**SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE**

**Fakulta elektrotechniky a informatiky**

Evidenčné číslo: FEI-184458-123409

# PORTÁL PRE ŠTUDENTOV S AI POMOCNÍKOM

**Bakalárska práca**

**2026** **Dmytrii Kaminskyi**

**SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE**

**Fakulta elektrotechniky a informatiky**

Evidenčné číslo: FEI-184458-123409

# PORTÁL PRE ŠTUDENTOV S AI POMOCNÍKOM

**Bakalárska práca**

| | |
|---|---|
| Študijný program: | informačné a komunikačné technológie |
| Študijný odbor: | informatika |
| Školiace pracovisko: | Ústav multimediálnych informačných a komunikačných technológií |
| Vedúci záverečnej práce: | doc. Ing. Radoslav Vargic, PhD. |

**2026** **Dmytrii Kaminskyi**

Slovenská technická univerzita v Bratislave
Ústav multimediálnych informačných a komunikačných technológií

Fakulta elektrotechniky a informatiky
Akademický rok: 2025/2026
Evidenčné číslo: FEI-184458-123409

# ZADANIE BAKALÁRSKEJ PRÁCE

| | |
|---|---|
| Študent: | **Dmytrii Kaminskyi** |
| ID študenta: | 123409 |
| Študijný program: | informačné a komunikačné technológie |
| Študijný odbor: | informatika |
| Vedúci práce: | doc. Ing. Radoslav Vargic, PhD. |
| Vedúci pracoviska: | doc. Ing. Radoslav Vargic, PhD. |
| Miesto vypracovania: | Ústav multimediálnych informačných a komunikačných technológií |

Názov práce: **Portál pre študentov s AI pomocníkom**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

Cieľom práce je navrhnúť a realizovať webovú aplikáciu portálu pre študentov využívajúcu RAG. Toto riešenie má poskytovať prehľad informácií o štúdiu spolu s AI pomocníkom. Modularita systému zabezpečí používateľom len tú funkcionalitu, ktorá im prináleží z hľadiska kompetencií v systéme. Riešenie by mohlo byť pokračovaním a rozvojom práce [1].

Úlohy:
1. Vykonajte analýzu aktuaĺneho riešenia a navrhnite zmeny
2. Realizujte a otestujte zmenené riešenie a zozbierajte skúsenosti z testovacej prevádzky.

Odporúčaná literatúra:
[1] M. Petkes, Portál pre študentov s AI pomocníkom , Bakalárska práca, FEI STU, 2025

Rozsah práce: 1 AH a maximálne 2 AH okrem prípadnej ďalšej technickej dokumentácie

| | |
|---|---|
| Termín odovzdania bakalárskej práce: | 05. 06. 2026 |
| Dátum schválenia zadania bakalárskej práce: | 14. 05. 2026 |
| Zadanie bakalárskej práce schválil: | doc. Ing. Radoslav Vargic, PhD. – garant študijného programu |

## Poďakovanie

Týmto by som sa chcel poďakovať vedúcemu mojej bakalárskej práce doc. Ing. Radoslavovi Vargicovi, PhD. za odborné vedenie, cenné rady, trpezlivosť a ústretový prístup počas konzultácií. Jeho pripomienky mi pomohli lepšie formulovať technické riešenie, spresniť štruktúru práce a sústrediť sa na podstatné časti realizovaného systému.

## Abstrakt

Táto bakalárska práca nadväzuje na predchádzajúcu prácu M. Petkesa [1] a realizuje analýzu, návrh zmien, implementáciu a testovanie portálu pre študentov s AI pomocníkom. Systém bol prepísaný do jednotného technologického zásobníka v jazyku TypeScript s klientskou časťou v Next.js a serverovou časťou v NestJS. Dátovú vrstvu tvorí PostgreSQL, Supabase Storage a vektorová databáza Qdrant. Hlavným prínosom práce je integrácia s akademickým informačným systémom STU, ktorá umožňuje synchronizáciu dokumentov predmetov za aktuálny semester. Nad synchronizovanými aj manuálne nahrávanými dokumentmi systém podporuje RAG asistenta, generovanie testovacích otázok a vyhodnocovanie odpovedí modelom gpt-4o-mini. Konverzačný asistent postavený na Flowise odpovedá na otázky k obsahu kurzov s využitím indexovaných dokumentov. Funkčnosť riešenia bola overená dvadsiatimi manuálnymi scenármi a orientačnými meraniami výkonu.

**Kľúčové slová:** portál pre študentov, RAG, veľké jazykové modely, akademický informačný systém, vektorová databáza, generovanie kvízov, Next.js, NestJS

## Abstract

This bachelor's thesis builds on the previous work by M. Petkes [1] and covers the analysis, design, implementation and testing of a student portal with an AI assistant. The system was rewritten into a unified TypeScript stack with a Next.js client and a NestJS server. The data layer is based on PostgreSQL, Supabase Storage and the Qdrant vector database. The main contribution is the integration with the STU Academic Information System, which enables synchronization of course documents for the current semester. Using synchronized and manually uploaded documents, the system supports a RAG assistant, quiz generation and answer evaluation with the gpt-4o-mini model. The Flowise-based conversational assistant answers course-related questions using indexed documents. The solution was verified through twenty manual test scenarios and indicative performance measurements.

**Keywords:** student portal, RAG, large language models, academic information system, vector database, quiz generation, Next.js, NestJS

## Obsah

Abstrakt ..... 5
Zoznam značiek a skratiek ..... 8
Úvod ..... 9
1 Súčasný stav a východiská riešenia ..... 10
1.1 Webové portály pre študentov na vysokých školách ..... 10
1.2 Veľké jazykové modely ..... 10
1.3 Generovanie rozšírené o vyhľadávanie (RAG) ..... 11
1.4 Riadenie prístupu na základe rolí ..... 13
1.5 Predchádzajúca práca ..... 13
2 Analýza aktuálneho riešenia a návrh zmien ..... 14
2.1 Architektúra systému a technologický zásobník ..... 14
2.2 Dátový model ..... 17
User a LoginSession ..... 17
Course, CourseEnrollment a materiály predmetov ..... 17
AisSession, AisSubject a AisDocument ..... 17
Stavy synchronizácie dokumentov AIS ..... 18
Pokrok v učení a kvízové pokusy ..... 18
2.3 Funkcionalita systému z pohľadu rolí ..... 18
Rola STUDENT ..... 18
Rola TEACHER ..... 19
2.4 Návrh na rozšírenie systému ..... 19
Doplnenie roly administrátora ..... 19
Zavedenie automatizovaných testov ..... 19
Dokončenie čiastočne implementovaných častí klientskej aplikácie ..... 20
Doplnenie infraštruktúrnej dokumentácie ..... 20
Obmedzenia súčasnej integrácie s AIS ..... 20
3 Návrh a realizácia upraveného riešenia ..... 21
3.1 Ciele upraveného riešenia ..... 21
3.2 Realizované zmeny v architektúre systému ..... 21

3.3 Realizácia integrácie s akademickým informačným systémom ..... 22
3.4 Realizácia spracovania dokumentov a RAG komponentu ..... 26
3.5 Realizácia kvízov a sledovania pokroku ..... 28
3.6 Realizácia konverzačného asistenta na platforme Flowise ..... 30
3.7 Bezpečnostné obmedzenia realizovaného riešenia ..... 33
3.8 Obmedzenia a ďalšie možné rozšírenia ..... 34
4 Testovanie a skúsenosti z testovacej prevádzky ..... 36
4.1 Metodika testovania ..... 36
4.2 Funkčné testovanie ..... 36
4.3 Výkonnostné merania ..... 37
4.4 Skúsenosti z testovacej prevádzky ..... 38
4.5 Zhrnutie testovania ..... 39
Záver ..... 41
Literatúra ..... 42
Deklarácia použitia umelej inteligencie ..... 43

## Zoznam značiek a skratiek

**AI** – umelá inteligencia (angl. Artificial Intelligence)

**AES** – symetrický šifrovací algoritmus, z angl. Advanced Encryption Standard

**AIS** – akademický informačný systém

**API** – rozhranie pre programovanie aplikácií (angl. Application Programming Interface)

**BM25** – štatistická vyhľadávacia funkcia (angl. Best Match 25)

**CSS** – kaskádové štýly (angl. Cascading Style Sheets)

**CUID** – kolízne odolný unikátny identifikátor (angl. Collision-resistant Unique Identifier)

**FEI** – Fakulta elektrotechniky a informatiky

**GCM** – režim autentifikovaného šifrovania, z angl. Galois/Counter Mode

**HTML** – hypertextový značkovací jazyk (angl. HyperText Markup Language)

**HTTP** – hypertextový prenosový protokol (angl. HyperText Transfer Protocol)

**HTTPS** – zabezpečený hypertextový prenosový protokol, z angl. HyperText Transfer Protocol Secure

**JSON** – javascriptový zápis objektov (angl. JavaScript Object Notation)

**LLM** – veľký jazykový model (angl. Large Language Model)

**NLP** – spracovanie prirodzeného jazyka (angl. Natural Language Processing)

**ORM** – objektovo-relačné mapovanie (angl. Object-Relational Mapping)

**PDF** – prenosný formát dokumentov (angl. Portable Document Format)

**RAG** – generovanie rozšírené o vyhľadávanie (angl. Retrieval-Augmented Generation)

**RBAC** – riadenie prístupu na základe rolí (angl. Role-Based Access Control)

**SHA** – bezpečná hashovacia funkcia (angl. Secure Hash Algorithm)

**SQL** – štruktúrovaný dotazovací jazyk (angl. Structured Query Language)

**STU** – Slovenská technická univerzita v Bratislave

**UI** – používateľské rozhranie (angl. User Interface)

**URL** – jednotný lokátor zdroja (angl. Uniform Resource Locator)

**UUID** – univerzálne jedinečný identifikátor (angl. Universally Unique Identifier)

# Úvod

Akademické portály na vysokých školách plnia predovšetkým evidenčnú funkciu: uchovávajú záznamy o rozvrhu, výsledkoch skúšok a harmonogramoch akademického roku. Väčšina z nich neposkytuje nástroje, ktoré by aktívne pomáhali študentom pri štúdiu obsahu predmetov. Veľké jazykové modely (ďalej LLM, z angl. Large Language Model) a technika generovania rozšíreného o vyhľadávanie (ďalej RAG, z angl. Retrieval-Augmented Generation) vytvárajú technologický základ, na ktorom je možné vybudovať inteligentného asistenta pracujúceho priamo nad inštitucionálne špecifickým obsahom.

Táto bakalárska práca nadväzuje na prácu M. Petkesa [1], v ktorej bol navrhnutý a implementovaný prvý prototyp študentského portálu s AI asistentom. Prototyp overil, že RAG je v tomto kontexte použiteľnou technikou, a identifikoval oblasti pre ďalší rozvoj: integráciu s akademickým informačným systémom, rolu vyučujúceho, mechanizmus sledovania pokroku a generovanie testovacích otázok. Aktuálne riešenie, označované ako PortalAI, všetky tieto oblasti adresuje a zároveň predstavuje úplné prepísanie systému do nového technologického zásobníka.

Práca naplňuje obe úlohy zo zadania. Prvá úloha — vykonanie analýzy aktuálneho riešenia a návrh zmien — je predmetom kapitoly 2. Druhá úloha — realizácia, testovanie a zozbieranie skúseností z testovacej prevádzky — je zdokumentovaná v kapitolách 3 a 4, pričom realizácia upraveného riešenia je opísaná v kapitole 3 a testovanie v kapitole 4. Kapitola 1 poskytuje prehľad súčasného stavu riešenej problematiky: teóriu LLM a embeddingových modelov, princíp RAG, model riadenia prístupu na základe rolí a charakteristiku predchádzajúcej práce [1].

# 1 Súčasný stav a východiská riešenia

## 1.1 Webové portály pre študentov na vysokých školách

Akademické portály pre vysokoškolských študentov poskytujú prístup k rozvrhom, výsledkom hodnotenia, harmonogramom akademického roku a k dokumentom predmetov. Slovenská technická univerzita v Bratislave prevádzkuje akademický informačný systém dostupný na adrese is.stuba.sk, prostredníctvom ktorého majú študenti prístup k materiálom predmetov, zápisom a výsledkom hodnotení.

Systémy správy výučby (angl. Learning Management System, ďalej LMS) ako Moodle alebo Canvas rozširujú evidenčnú funkcionalitu o prvky podporujúce učenie: odovzdávanie zadaní, diskusné fóra a sledovanie aktivít. Tieto systémy sú spravidla nasadené oddelene od evidenčných systémov a vyžadujú manuálne vkladanie obsahu vyučujúcimi.

Komerčné nástroje s generatívnou umelou inteligenciou — ako Khanmigo od Khan Academy alebo asistent integrovaný v platforme Coursera — ponúkajú interakciu v prirodzenom jazyku, no sú viazané na obsah konkrétnej platformy a nedokážu sa prispôsobiť materiálom špecifickým pre danú inštitúciu. Práca [1] identifikovala tento priestor ako motiváciu pre vývoj vlastného portálu pracujúceho s dokumentmi z inštitucionálneho akademického systému.

## 1.2 Veľké jazykové modely

Veľké jazykové modely sú neurónové siete trénované na rozsiahlych textových korpusoch. Ich základnou úlohou je modelovanie pravdepodobnostného rozdelenia nasledujúceho tokenu v sekvencii, čo im umožňuje generovať koherentný text, odpovedať na otázky a vykonávať úlohy spracovania prirodzeného jazyka.

Kľúčovým architektonickým prínosom bolo zavedenie modelu Transformer [3] v roku 2017. Mechanizmus pozornosti (angl. attention) umožnil paralelnú väzbu medzi ľubovoľnými tokenmi v sekvencii bez obmedzení, ktoré mali predchádzajúce rekurentné architektúry. Na tomto základe vznikli modely BERT, GPT-2, GPT-3 a ďalšie. Typický postup zahŕňa fázu predtrénovania na veľkých textových kolekciách a fázu doladenia (angl. fine-tuning) pre konkrétnu doménu alebo úlohu.

Pre vzdelávacie aplikácie sú relevantné dve obmedzenia LLM. Prvým je halucinácia: model môže generovať vecne nepresné, no jazykovo plynulé tvrdenia, pretože optimalizuje pravdepodobnosť nasledujúceho tokenu, nie faktickú správnosť. Druhým je zastaranie znalostí: model pracuje len s informáciami prítomnými v trénovacích dátach ku dňu uzávierky a nedokáže pracovať s inštitucionálne špecifickým obsahom. Na obidva problémy reaguje prístup RAG [2, 4].

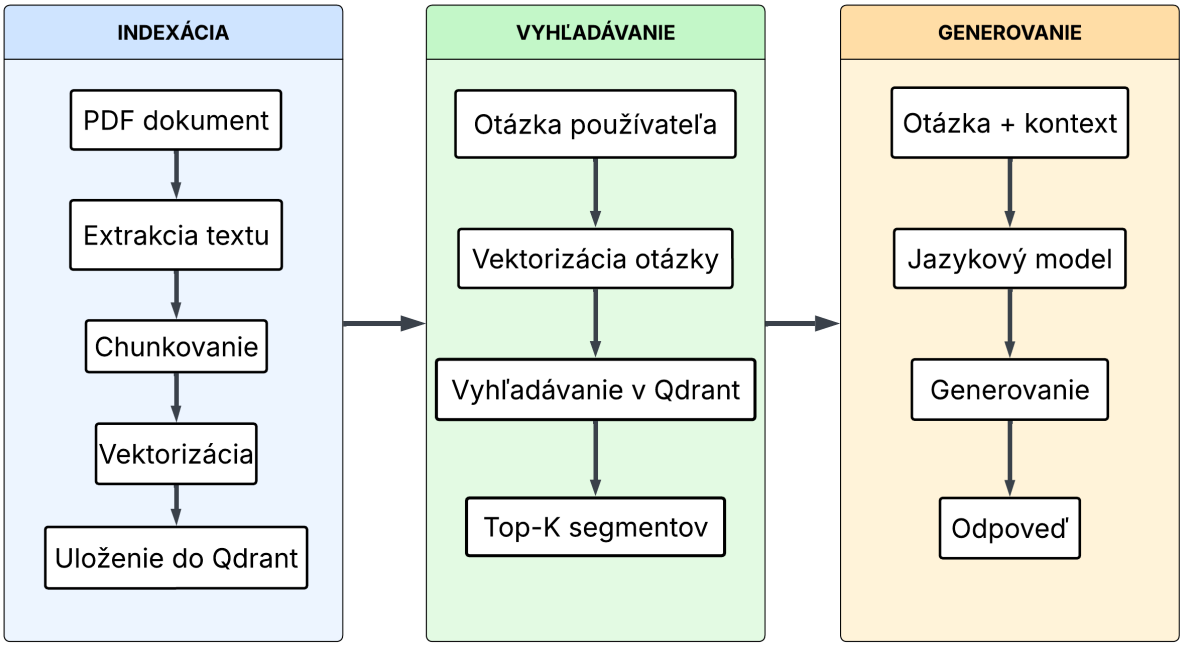
Pre technickú realizáciu RAG sú nevyhnutné embeddingové modely. Tieto modely transformujú text na číselné vektory v mnohorozmernom priestore tak, aby si sémanticky príbuzné texty zodpovedali blízkymi vektormi. Model text-embedding-3-small od spoločnosti OpenAI, použitý v analyzovanom systéme, produkuje vektory o rozmere 1 536 dimenzií. Text-embedding-3-small predstavuje kompromis medzi kvalitou a výpočtovou efektívnosťou v rámci rady embeddingových modelov OpenAI [9].

Výskum zameraný na využitie LLM vo vzdelávaní zaznamenal výrazný rast po roku 2022 [5]. Medzi skúmané prínosy patrí dostupnosť pomoci nezávislá od pracovného času vyučujúcich a automatizované generovanie cvičných otázok. Medzi identifikované riziká patrí šírenie nepresných informácií a potenciálna závislosť na automatizovanej pomoci bez kritického posúdenia výstupov.

## 1.3 Generovanie rozšírené o vyhľadávanie (RAG)

Generovanie rozšírené o vyhľadávanie kombinuje generatívne schopnosti LLM so schopnosťou vyhľadávať v indexovaných dokumentových kolekciách [2]. Pred každým generovaním odpovede systém vyhľadá úseky textu relevantné k otázke a zaradí ich do kontextu modelu. Tým sa znižuje pravdepodobnosť halucinácie, pretože model môže svoju odpoveď oprieť o konkrétne dohľadané pasáže. Zároveň odpadá potreba pretrénovania modelu pri zmene obsahu.

Implementácia RAG pozostáva z troch fáz. V prvej fáze — indexácii — sa dokumenty rozdeľujú na menšie úseky. Každý úsek sa transformuje na číselný vektorový záznam prostredníctvom embeddingového modelu a uloží sa do vektorovej databázy. V druhej fáze — vyhľadávaní — sa otázka používateľa rovnako transformuje na vektor a vyhľadajú sa najbližšie segmenty podľa kosínusovej podobnosti. V tretej fáze — generovaní — sa nájdené segmenty spolu s otázkou vložia do kontextu LLM, ktorý vygeneruje odpoveď. Schéma všetkých troch fáz je znázornená na Obr. 1.




*Obr. 1: Schéma troch fáz RAG: indexácia dokumentov, vyhľadávanie relevantných segmentov a generovanie odpovede*

V systéme PortalAI to znamená, že keď študent položí otázku k obsahu kurzu alebo predmetu z AIS, systém neodpovedá iba na základe všeobecných znalostí jazykového modelu. Najprv vyhľadá v indexovaných dokumentoch zvoleného kurzu alebo predmetu a najrelevantnejšie textové segmenty vloží do kontextu modelu. Asistent tak môže odpovedať s odkazom na konkrétne zdrojové materiály. Tento postup znižuje riziko halucinácií a umožňuje spätne dohľadať, z ktorých dokumentov odpoveď vychádza.

Kľúčovými nastaviteľnými parametrami RAG sú veľkosť segmentu, prekryv susedných segmentov a počet vyhľadávaných segmentov. Menšie segmenty poskytujú presnejší výsledok vyhľadávania, ale strácajú kontext; väčšie segmenty zachovávajú kontext, no vyhľadávač môže byť menej presný. Výber hodnôt závisí od povahy dokumentov a vyžaduje experimentovanie [4].

Jednoduchá implementácia hustého vyhľadávania nemusí byť dostatočná pri otázkach obsahujúcich špecifické technické pojmy alebo vlastné mená. Hybridné vyhľadávanie — kombinácia hustého vektorového vyhľadávania a riedeho vyhľadávania na základe kľúčových slov (napr. BM25) — môže v takýchto prípadoch priniesť lepšie výsledky [4].

Kvalitu RAG systému ovplyvňujú dva oddeliteľné aspekty: presnosť vyhľadávania a kvalita generovania. Chyby vyhľadávania model nedokáže kompenzovať: ak relevantný segment v kontexte chýba, model buď halucinuje, alebo prizná, že informácia nie je k dispozícii. Preto je voľba parametrov segmentácie, embeddingového modelu a počtu vyhľadávaných segmentov rozhodujúca.

## 1.4 Riadenie prístupu na základe rolí

Riadenie prístupu na základe rolí (RBAC) je model bezpečnostnej politiky, v ktorom sa oprávnenia prideľujú roliam, nie jednotlivým používateľom priamo. Každý používateľ má priradenú rolu a každá rola definuje množinu povolených operácií nad systémovými zdrojmi. RBAC zjednodušuje správu prístupových práv v systémoch s väčším počtom používateľov.

V kontexte vzdelávacích portálov je RBAC prirodzeným nástrojom na oddelenie funkcionalít dostupných rôznym skupinám: študenti môžu čítať materiály a interagovať s asistentom, vyučujúci môžu nahrávať materiály a spravovať predmety. Správne implementovaný model RBAC predchádza neautorizovanému prístupu k dátam iných používateľov.

V prostredí softvérových rámcov ako NestJS je RBAC zvyčajne centralizovaný v strážcoch (angl. guards) alebo dekorátoroch, ktoré sa vyhodnocujú pred spracovaním každej prichádzajúcej požiadavky. Rovnomerné uplatňovanie politiky prístupu na všetkých trasách je predpokladom pre spoľahlivé oddelenie rolí v praxi.

## 1.5 Predchádzajúca práca

Predchádzajúca bakalárska práca M. Petkesa [1] navrhla a implementovala prvý prototyp študentského portálu s AI asistentom. Systém bol postavený na softvérovom rámci Svelte pre klientskú časť a jazyku Python s rámcom FastAPI pre serverovú časť. Na autentifikáciu a ukladanie dát bola použitá cloudová platforma Firebase (Firestore a Firebase Storage). Vektorová databáza ChromaDB slúžila na ukladanie embeddingov. Jazykové modely aj embeddingy pochádzali od spoločnosti OpenAI (GPT-3.5 Turbo a príslušný embeddingový model). Na orchestráciu RAG bol použitý rámec LangChain.

Systém umožňoval nahrávanie dokumentov, ich vektorizáciu a kladenie otázok k obsahu prostredníctvom chatového rozhrania. Práca [1] overila niekoľko stratégií segmentácie textu: rekurzívnu, sémantickú a špeciálnu pre informačné listy predmetov. Pre spracovanie informačných listov predmetov bol sémantický postup nahradený prístupom rešpektujúcim hranice dokumentu ako celku, čo výrazne zlepšilo presnosť odpovedí.

Práca [1] testovala lokálne modely Llama 3.2 a Mistral 7B ako alternatívy k modelom OpenAI. Oba lokálne modely vykazovali nižšiu kvalitu odpovedí v slovenčine a výrazne dlhší čas spracovania. Preto bol pre produkčnú variantu zvolený model GPT-3.5 Turbo.

Práca [1] identifikovala oblasti pre ďalší rozvoj: integráciu s AIS, samostatnú rolu vyučujúceho, sledovanie pokroku v učení a automatické generovanie testovacích otázok. Potenciál prechodu na TypeScript naprieč celým systémom bol identifikovaný ako príležitosť pre lepšiu udržiavateľnosť. Tieto oblasti sú adresované v systéme PortalAI opísanom v nasledujúcich kapitolách.

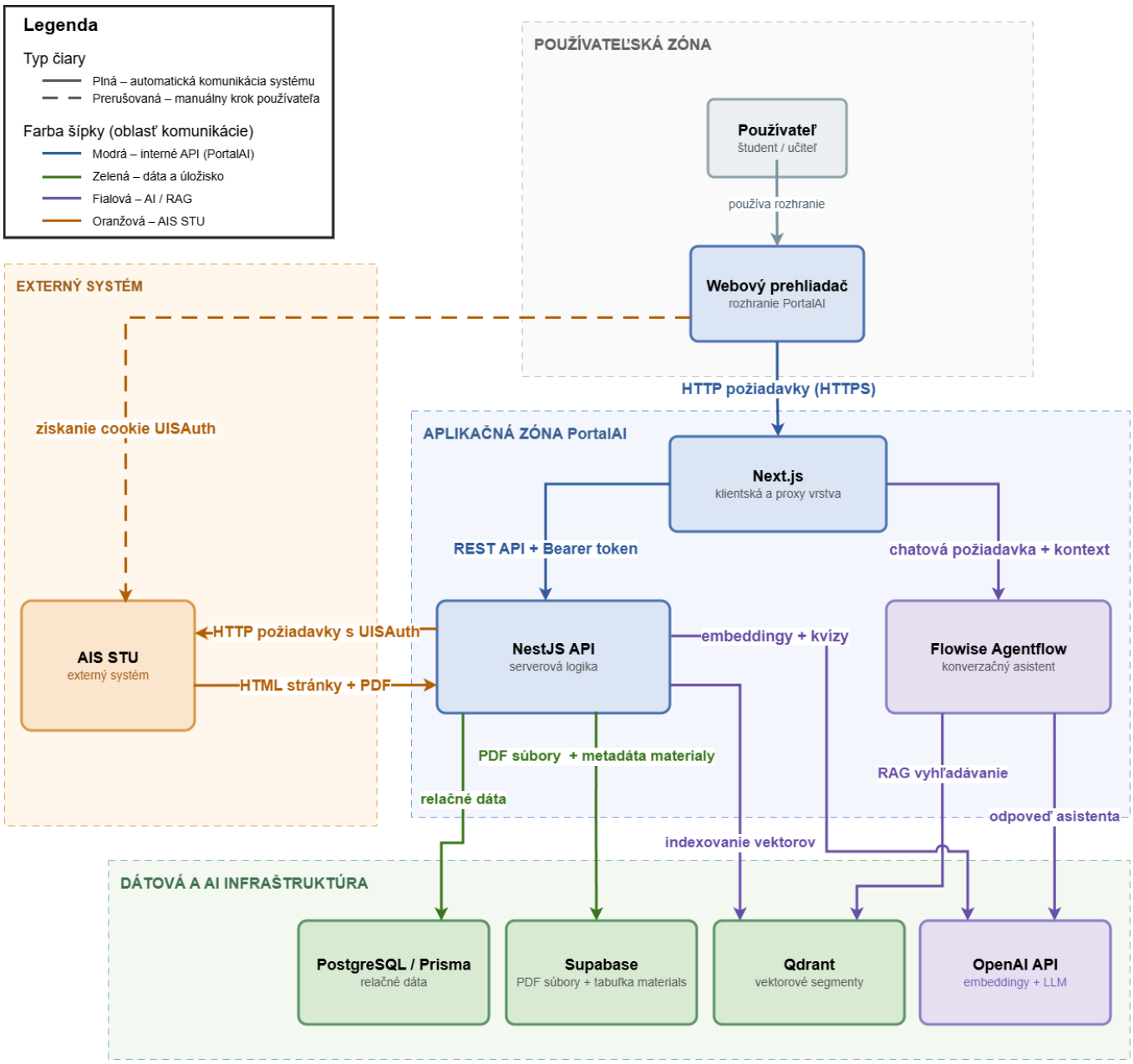
# 2 Analýza aktuálneho riešenia a návrh zmien

Nasledujúca kapitola opisuje aktuálny stav riešenia PortalAI z pohľadu architektúry, dátového modelu a dostupných funkcií. Na základe tejto analýzy sú následne pomenované oblasti ďalšieho rozšírenia systému.

## 2.1 Architektúra systému a technologický zásobník

Diagram architektúry systému PortalAI s vyznačenými komponentmi a závislosťami je zobrazený na Obr. 2.

Systém je rozdelený do dvoch hlavných komponentov tvoriacich monorepozitár. Prvým je klientská časť (adresár portal-ai) implementovaná v softvérovom rámci Next.js vo verzii 15 [7] s knižnicou React vo verzii 19. Druhým je serverová časť (adresár core-api) implementovaná v softvérovom rámci NestJS vo verzii 11 [6]. Oba komponenty sú napísané v jazyku TypeScript, čo zaručuje konzistentnú typovú kontrolu naprieč celým systémom — zásadný posun oproti dvojjazyčnému zásobníku predchádzajúcej práce [1].




*Obr. 2: Diagram architektúry systému PortalAI: klientská časť (Next.js), serverová časť (NestJS), Flowise, PostgreSQL, Qdrant, Supabase, OpenAI API*

Klientská časť plní funkciu proxy vrstvy medzi webovým prehliadačom a serverovou časťou. Trasovanie URL adries je riešené prostredníctvom App Routera Next.js; každá skupina trás zodpovedá jednej používateľskej sekcii (student, teacher, auth). Klientská časť obsahuje vlastné HTTP trasy, ktoré preposielajú požiadavky na serverovú časť s autentifikačným tokenom extrahovaným zo súboru cookie a pripojeným ako hlavička Authorization: Bearer. Okrem serverovej časti proxy vrstva preposiela chatové požiadavky aj do externej inštancie Flowise, ktorá je podrobnejšie opísaná v časti 3.6.

Serverová časť je rozdelená do šiestich modulov: auth (autentifikácia a správa relácií), courses (kurzy a zápisy), materials (materiály predmetov, nahrávanie, vektorizácia), ais (integrácia s AIS, synchronizácia dokumentov, generovanie kvízov a sledovanie pokroku pre predmety z AIS), progress (generovanie kvízov a sledovanie pokroku pre bežné kurzy

vytvorené vyučujúcimi) a prisma (databázové pripojenie). Väčšina aplikačných modulov obsahuje radič, servisnú vrstvu a definíciu modulu podľa konvencií NestJS. Výnimkou je PrismaModule, ktorý slúži ako infraštruktúrny modul poskytujúci databázovú službu pre ostatné časti systému — neobsahuje vlastný radič, iba PrismaService. Globálna stráž AuthGuard overuje platnosť tokenu relácie pri každej prichádzajúcej požiadavke; verejné trasy sú označené dekorátorom @SkipAuth().

Modul progress (ProgressModule) je samostatným modulom s vlastným radičom (ProgressController) a servisnou vrstvou (ProgressService). Zabezpečuje kvízovú funkcionalitu pre bežné kurzy vytvorené vyučujúcimi: generovanie testovacích otázok z materiálov kurzu, sémantické hodnotenie otvorených odpovedí, ukladanie pokusov (QuizAttempt, QuizAnswer) a prepočet agregovaného pokroku študenta (StudentCourseProgress). Systém tak obsahuje dve paralelné implementácie kvíz/pokrok: AisModule pre predmety z AIS (AisQuizAttempt, AisQuizAnswer, AisSubjectProgress) a ProgressModule pre bežné kurzy (QuizAttempt, QuizAnswer, StudentCourseProgress).

Úplný prehľad použitých technológií a ich verzií uvádza Tab. 1.

*Tab. 1: Prehľad použitých technológií*

| **Komponent** | **Technológia** | **Verzia / poznámka** |
|---|---|---|
| Klientská časť | Next.js | 15 |
| Klientská časť | React | 19 |
| Klientská časť | TypeScript | naprieč celým systémom |
| Klientská časť | Tailwind CSS, Radix UI | štýlovanie a komponenty |
| Klientská časť | vlastná komponenta React | konverzačné rozhranie s proxy trasou do Flowise API |
| Serverová časť | NestJS | 11 |
| Serverová časť | Prisma ORM | 7 — prístup k PostgreSQL |
| Serverová časť | Multer | príjem nahrávaných súborov |
| Serverová časť | Cheerio | analýza HTML odpovedí AIS |
| Serverová časť | pdf-parse | extrakcia textu z PDF |
| Serverová časť | Playwright | automatizácia prehliadača (iba vývojové prostredie) |
| Databáza / úložisko | PostgreSQL | primárna relačná databáza |
| Databáza / úložisko | Qdrant | vektorová databáza, kolekcia portalai_materials |
| Databáza / úložisko | Supabase Storage | fyzické ukladanie súborov (bucket portal-materials) |
| Databáza / úložisko | Supabase DB | tabuľka materials (mimo Prisma schémy) |
| Umelá inteligencia | OpenAI text-embedding-3-small | 1 536-rozmerné vektory |
| Umelá inteligencia | OpenAI gpt-4o-mini | generovanie kvízov, hodnotenie odpovedí |

| | | |
|---|---|---|
| Umelá inteligencia | Flowise | konverzačný asistent (externá inštancia) |
| Bezpečnosť | bcrypt | hashovanie hesiel, 12 iterácií |
| Bezpečnosť | Node.js crypto (AES-256-GCM) | šifrovanie cookie AIS, generovanie tokenov |

## 2.2 Dátový model

Dátový model je definovaný v schéme Prisma a spravovaný prostredníctvom Prisma ORM. Metadáta materiálov predmetov vyučujúceho sú uložené oddelene v tabuľke materials prístupnej priamo cez klienta Supabase — táto tabuľka nie je súčasťou schémy Prisma. Dátový model oddeľuje tri druhy údajov: používateľov a ich prihlasovacie relácie, kurzy vytvorené vyučujúcimi spolu s ich materiálmi a predmety prevzaté z AIS so synchronizovanými dokumentmi. Toto rozdelenie zároveň vysvetľuje, prečo systém rozlišuje pojmy kurz (vytvorený vyučujúcim) a predmet (prevzatý z AIS).

***User a LoginSession***

Entita User uchováva základné údaje o používateľovi: unikátny identifikátor (CUID), e-mail, voliteľné používateľské meno (username), meno, hash hesla a rolu (STUDENT alebo TEACHER). Entita LoginSession reprezentuje aktívnu reláciu: obsahuje token, referenciu na používateľa a čas vypršania platnosti. Ukončené relácie sa odstraňujú priebežne pri každom prihlásení.

***Course, CourseEnrollment a materiály predmetov***

Entita Course reprezentuje kurz vytvorený vyučujúcim: kód, názov, popis a referenciu na vyučujúceho. Entita CourseEnrollment spravuje väzby medzi kurzmi a zapísanými študentmi s kompozitnou unikátnou podmienkou courseId + studentId. Metadáta fyzických súborov materiálov (originálny názov, cesta v úložisku, počet segmentov, čas indexácie) sú uložené v tabuľke materials v Supabase databáze. Fyzické súbory sú uložené v Supabase Storage a vektorové záznamy segmentov v Qdrant.

***AisSession, AisSubject a AisDocument***

Entita AisSession uchováva šifrovaný autentifikačný súbor cookie pre daného používateľa, príznak aktivity, čas poslednej synchronizácie a atribút activeAisSubjectId — identifikátor predmetu, ku ktorému prebehla posledná úspešná synchronizácia dokumentov. Tento atribút slúži ako autoritatívny zdroj pravdy na strane servera o práve aktívnom predmete AIS.

Entita AisSubject reprezentuje predmet v AIS priradený konkrétnemu používateľovi; obsahuje kód predmetu, názov predmetu, identifikátor priečinka semestra, semestrálny popis a atribút chatCourseId — unikátne UUID pridelené každému predmetu pri jeho prvom vytvorení. Toto UUID sa zapisuje ako hodnota poľa courseId v metadátach

vektorov v Qdrant a slúži na smerovanie konverzačného asistenta Flowise k správnemu predmetu. Nejde pritom o kurz v relačnej databáze, ale o interný identifikátor, ktorý asistenta nasmeruje na správny predmet z AIS. Entita AisDocument sleduje jednotlivé súbory zo zoznamu AIS a ich stav synchronizácie.

***Stavy synchronizácie dokumentov AIS***

Atribút syncStatus v entite AisDocument nadobúda nasledujúce hodnoty: discovered (dokument objavený na AIS, zástupný záznam vytvorený), uploaded (prechodný stav — súbor bol stiahnutý a uložený do Supabase, ale ešte nie je zaindexovaný), failed_upload (zlyhalo stiahnutie alebo uloženie do Supabase), unsupported (nepodporovaný formát súboru — nie PDF), failed_extraction (chyba extrakcie textu z PDF), no_text (PDF neobsahoval extrahovateľný text), failed_indexing (chyba pri vektorizácii alebo ukladaní do Qdrant), indexed (dokument úspešne spracovaný a zaindexovaný). Záverečnými stavmi sú: discovered, failed_upload, unsupported, failed_extraction, no_text, failed_indexing a indexed. Stav uploaded je prechodný: systém ho zapíše po úspešnom nahraní súboru do Supabase, ešte pred spustením extrakcie textu a indexácie. Ak spracovanie v tomto bode skončí chybou alebo je prerušené, záznam zostane v stave uploaded — čo signalizuje neúplne spracovaný dokument.

***Pokrok v učení a kvízové pokusy***

Entita StudentCourseProgress (a analogicky AisSubjectProgress pre predmety z AIS) uchováva agregovaný pokrok študenta: odhadnutú úroveň (beginner / intermediate / advanced), vážený priemer skóre podľa počtu otázok v jednotlivých pokusoch, počty pokusov a otázok a JSON polia slabých (weakTopics) a silných tém (strongTopics). Témy sú identifikované podľa názvu zdrojového súboru. Slabé témy zodpovedajú zdrojovým súborom s presnosťou pod 50 %, silné témy s presnosťou aspoň 80 %. Entita QuizAttempt zaznamenáva každý pokus o kvíz spolu so skóre a typom otázok. Entita QuizAnswer uchováva jednotlivé odpovede, výsledok AI hodnotenia pre otvorené otázky a JSON polia prítomných a chýbajúcich konceptov zo spätnej väzby.

Väzby medzi entitami sú navrhnuté s kaskádovým mazaním. Vymazanie záznamu AisSubject automaticky vymaže všetky súvisiace záznamy AisDocument a AisSubjectProgress. Tento prístup predchádza vzniku osirelých záznamov bez potreby explicitnej logiky mazania na aplikačnej úrovni.

## 2.3 Funkcionalita systému z pohľadu rolí

***Rola STUDENT***

Študent má prístup k nasledujúcim funkciám: registrácia a prihlásenie, pripojenie k AIS prostredníctvom autentifikačného súboru cookie, výber semestra a synchronizácia

dokumentov predmetov z AIS, interakcia s konverzačným asistentom a prezeranie prehľadu pokroku v učení (dosiahnuté skóre, slabé a silné témy, odhadnutá úroveň).

Kvízová funkcionalita je realizovaná v dvoch paralelných implementáciách podľa zdroja materiálov. Pre bežné kurzy vytvorené vyučujúcimi zabezpečuje generovanie testovacích otázok, hodnotenie odpovedí a sledovanie pokroku modul ProgressModule. Pre predmety z AIS plní rovnakú funkciu modul AisModule. Oba moduly používajú model gpt-4o-mini na generovanie otázok aj sémantické hodnotenie otvorených odpovedí, no uchovávajú výsledky v oddelených entitách databázy.

***Rola TEACHER***

Vyučujúci má prístup k nasledujúcim funkciám: vytváranie kurzov, nahrávanie PDF materiálov k predmetu, prezeranie zoznamu zapísaných študentov, správa zápisu, generovanie testov z materiálov predmetu. Materiály nahrané vyučujúcim zdieľajú s dokumentmi z AIS najmä fázy extrakcie textu, segmentácie, vektorizácie a uloženia do Qdrant, no líšia sa poradím spracovania a spôsobom evidencie stavov.

## 2.4 Návrh na rozšírenie systému

Na základe analýzy aktuálneho riešenia, predchádzajúcej práce [1] a zdrojového kódu projektu boli navrhnuté konkrétne oblasti rozšírenia systému, ktoré tvoria základ pre realizáciu zmien opísaných v kapitole 3.

Rozšírenia realizované v rámci tejto práce, najmä integrácia s AIS, doplnenie roly vyučujúceho, generovanie kvízov, sledovanie pokroku a zapojenie konverzačného asistenta, sú opísané v kapitole 3. Nasledujúce oblasti predstavujú návrhy ďalšieho rozšírenia a zároveň hranice aktuálneho prototypu.

***Doplnenie roly administrátora***

V systéme nie je implementovaná rola správcu systému. Správa naprieč všetkými používateľmi a predmetmi nie je súčasťou aktuálnej implementácie, čo obmedzuje možnosti dohľadu nad prevádzkou systému.

***Zavedenie automatizovaných testov***

V repozitári sa nachádzajú iba základné šablónové súbory testov vygenerované alebo ponechané počas vývoja, napríklad app.controller.spec.ts, auth.controller.spec.ts a auth.service.spec.ts. Tieto testy nepokrývajú reálnu aplikačnú logiku ani integračné scenáre systému. Absencia testov sťažuje overovanie správnosti pri každej zmene kódu a zvyšuje riziko zavedenia regresných chýb.

***Dokončenie čiastočne implementovaných častí klientskej aplikácie***

Stránka /settings je implementovaná a umožňuje nastavenie lokálnych preferencií používateľského rozhrania, avšak neposkytuje úplnú editáciu serverového profilu (napríklad zmenu e-mailovej adresy alebo mena). Stránka /help je implementovaná ako prehľad funkcionality systému a sekcia najčastejších otázok (FAQ), avšak neobsahuje podrobného sprievodcu prvým prihlásením do AIS.

***Doplnenie infraštruktúrnej dokumentácie***

Repozitár neobsahuje jednotný konfiguračný súbor docker-compose.yml pre spustenie celého systému PortalAI ani úplnú infraštruktúrnu dokumentáciu nasadenia. Systém závisí od externej inštancie Flowise, pričom jej konfigurácia nie je súčasťou repozitára. To sťažuje opakovateľné nasadenie v novom prostredí.

***Obmedzenia súčasnej integrácie s AIS***

Integrácia s AIS je navrhnutá ako prototypové kompromisné riešenie závislé od manuálne skopírovaného autentifikačného súboru cookie. Toto riešenie nie je vhodné pre dlhodobú produkčnú prevádzku. Podrobná analýza bezpečnostných obmedzení a návrh zlepšení sú predmetom kapitoly 3.

# 3 Návrh a realizácia upraveného riešenia

Táto kapitola popisuje návrh a implementáciu zmien systému PortalAI, ktoré vychádzajú z analýzy predchádzajúcej práce [1] zhrnutej v kapitole 2. Systém PortalAI je funkčný prototyp — všetky opísané funkcie sú implementované a overené manuálnym testovaním zdokumentovaným v kapitole 4, avšak systém nie je dimenzovaný pre produkčné nasadenie.

## 3.1 Ciele upraveného riešenia

Oproti predchádzajúcej práci [1] boli stanovené tieto realizačné ciele: zjednotenie technologického zásobníka do jazyka TypeScript, nahradenie Firebase a ChromaDB dátovou vrstvou s relačným modelom, riadenými migráciami a oddeleným vektorovým úložiskom (PostgreSQL, Qdrant, Supabase), implementácia integrácie s akademickým informačným systémom STU, doplnenie roly vyučujúceho s možnosťou správy kurzov a materiálov, automatické generovanie testovacích otázok s okamžitou spätnou väzbou a zavedenie mechanizmu sledovania pokroku v učení na úrovni tém zdrojových dokumentov.

Tieto ciele priamo vychádzajú z nedostatkov identifikovaných v predchádzajúcej práci [1] a z požiadaviek zadania. Výsledkom je funkčný systém overený v lokálnom testovacom prostredí, ktorého merateľné výsledky sú zdokumentované v kapitole 4.

## 3.2 Realizované zmeny v architektúre systému

Najvýraznejšou zmenou oproti predchádzajúcej práci [1] je úplné prepísanie systému. Klientská časť prešla zo Svelte na Next.js 15 s Reactom 19; serverová časť prešla z Pythonu a FastAPI na TypeScript a NestJS 11. Oba komponenty sú teraz písané v rovnakom jazyku, čo zjednodušuje udržiavanie a umožňuje zdieľanie typových definícií medzi vrstvami.

Dátová vrstva bola nahradená zásadne: Firebase Firestore a Firebase Storage nahradilo PostgreSQL (prístup cez Prisma ORM [10]), Supabase Storage a Supabase databáza [11]. ChromaDB ako vektorová databáza bola nahradená Qdrantom [8]. Táto zmena priniesla silnejší relačný dátový model s migráciami spravovanými Prismou, a zároveň oddelenie vektorového úložiska od primárnej databázy.

Tok autentifikácie prebieha nasledovne. Po úspešnom prihlásení serverová časť vygeneruje token vo formáte local_{userId}_{uuid}, kde uuid je kryptograficky bezpečné UUID generované funkciou randomUUID() z modulu Node.js crypto. Token sa uloží do tabuľky LoginSession s platnosťou sedem dní a vráti sa klientskej časti. Tá ho uloží do súboru cookie s príznakom HttpOnly, čím sa zamedzí jeho prístupnosti z kódu JavaScript bežiaceho v prehliadači. Stráž AuthGuard overí token v databáze a neplatná relácia je odmietnutá kódom HTTP 401. Heslá sú hashované algoritmom bcrypt s 12 iteráciami.

Po prihlásení klientská strana pracuje s tromi súbormi cookie. Súbor access_token je nastavený s príznakom HttpOnly a slúži na autentifikáciu API požiadaviek — JavaScript v prehliadači k nemu nemá priamy prístup. Súbory user_role a user_id sú nastavené bez príznaku HttpOnly: slúžia výhradne na smerovanie na správnu klientskú stránku podľa roly a na identifikáciu používateľa v proxy vrstve klientskej časti. Tieto hodnoty nie sú tajné v kryptografickom zmysle — rola a identifikátor používateľa sú dostupné z iných odpovedí API.

Konverzačný asistent je integrovaný prostredníctvom externej inštancie Flowise, ktorá zabezpečuje orchestráciu Agentflow; klientská strana využíva vlastnú React komponentu konverzačného rozhrania. Proxy trasa v Next.js obohacuje chatové požiadavky smerované do Flowise o kontextové premenné courseId, userId a userRole. Flowise Agentflow tieto premenné využíva na vyhľadávanie v Qdrant v kontexte správneho kurzu alebo predmetu AIS a na zachovanie oddeleného prístupu medzi používateľmi. Konverzácia sa smeruje k správnemu predmetu na základe hodnoty chatCourseId, ktorá je unikátnym UUID priradeným každému predmetu AIS.

## 3.3 Realizácia integrácie s akademickým informačným systémom

Integrácia s AIS STU (is.stuba.sk) je najvýznamnejším novým prínosom oproti predchádzajúcej práci [1]. Počas vývoja nebola nájdená verejne dostupná dokumentácia k programovému rozhraniu tohto systému, ktoré by umožňovalo štandardizovaný prístup k dátam. Realizovaný prístup preto využíva sieťovú komunikáciu prostredníctvom autentifikačných súborov cookie a analýzu HTML odpovedí servera.

Integrácia s AIS prebieha v dvoch nadväzujúcich fázach. V prvej fáze — pri pripojení AIS relácie — používateľ vloží hodnotu cookie UISAuth, ktorú získa z aktívnej relácie prehliadača pri prihlásení do AIS. Táto hodnota sa v serverovej časti zašifruje algoritmom AES-256-GCM z modulu Node.js crypto a uloží do PostgreSQL v entite AisSession. Režim GCM (Galois/Counter Mode) poskytuje autentifikované šifrovanie s autentifikačnou značkou (authentication tag), vďaka čomu je možné odhaliť akúkoľvek neautorizovanú zmenu uloženej šifrovanej hodnoty. V druhej fáze — pri spustení synchronizácie — sa zašifrovaná hodnota cookie UISAuth dešifruje a použije pri HTTP požiadavkách na server AIS; systém nastavuje hlavičky Cookie: UISAuth={hodnota} a User-Agent: Mozilla/5.0.

Súčasťou prvej fázy je overenie platnosti relácie AIS, ktoré prebieha načítaním indexovej stránky dokumentového servera (/auth/dok_server/index.pl). Systém vyhodnotí vrátený HTML obsah a hľadá v ňom markery ako "Dokumentový server", "Prihlásený:" a "Odhlásenie". Ak tieto markery chýbajú, relácia je považovaná za vypršanú alebo neplatnú: záznam AisSession sa označí ako neaktívny a používateľovi sa zobrazí výzva na opätovné pripojenie. Analýza HTML obsahu stránok AIS prebieha knižnicou Cheerio.

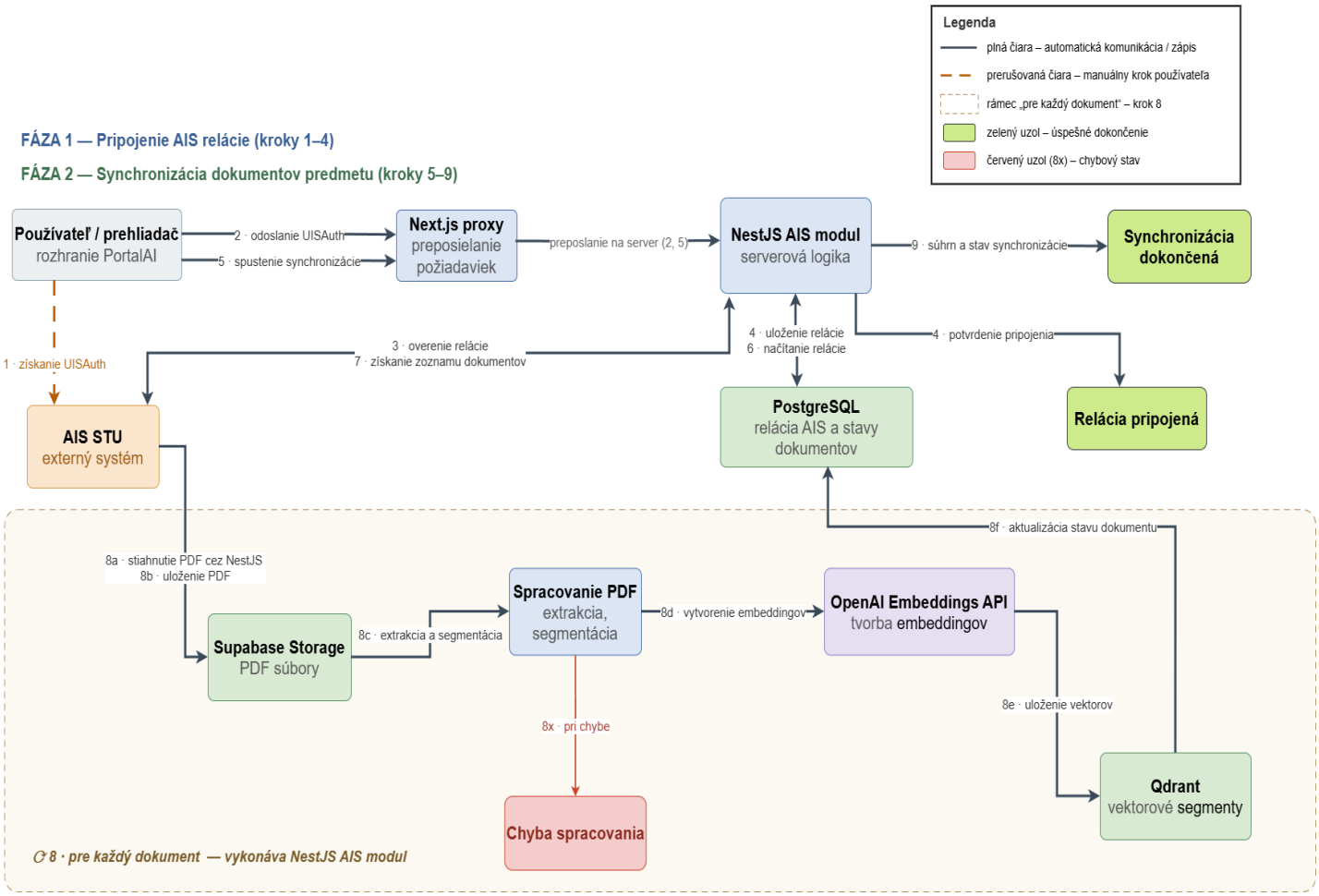
Identifikácia priečinka so zoznamom predmetov je implementovaná so štyrmi zálohovacími stratégiami v poradí priority: presná zhoda textového označenia priečinka s hodnotami Predmety / Courses / Subjects (stratégia 0), vyhľadanie najhlbšieho uzla v HTML strome s potomkami vo forme semestrálnych odkazov (stratégia A) a dve ďalšie zálohovacie stratégie pre prípad, že predchádzajúce zlyhajú. Táto viacnásobná zálohovacia štruktúra zmierňuje závislosť od konkrétnej HTML štruktúry stránky AIS, hoci ju neodstraňuje úplne.

Semester sa overuje regulárnym výrazom, ktorý akceptuje označenia vo formáte ZS RRRR/RRRR alebo LS RRRR/RRRR (prípadne anglické ekvivalenty WS, SS). Semester sa určuje automaticky podľa kalendárneho mesiaca: september až december a január zodpovedajú zimnému semestru (ZS), február až august letnému semestru (LS).

Synchronizácia dokumentov predmetu je implementovaná ako úplné nahradenie: pred každou synchronizáciou sa vymažú všetky databázové záznamy aj vektorové body daného predmetu z Qdrantu a vytvárajú sa znovu od základu. Tým je zaistená konzistencia s aktuálnym stavom na AIS — dokumenty odstránené z AIS zmiznú aj z lokálneho zrkadla po ďalšej synchronizácii. Filter pri mazaní sa striktne obmedzuje na trojicu sourceType, userId a subjectId, čím sa nikdy nedotknú vektorov iných používateľov alebo iných predmetov. Stiahnuté PDF súbory sa ukladajú do Supabase Storage; relačné záznamy predmetov, dokumentov a stavy synchronizácie (syncStatus) sa zapisujú do PostgreSQL; textové segmenty vektorizované pre RAG komponent sa ukladajú do Qdrant. Podrobný postup spracovania dokumentov je opísaný v kapitole 3.4.

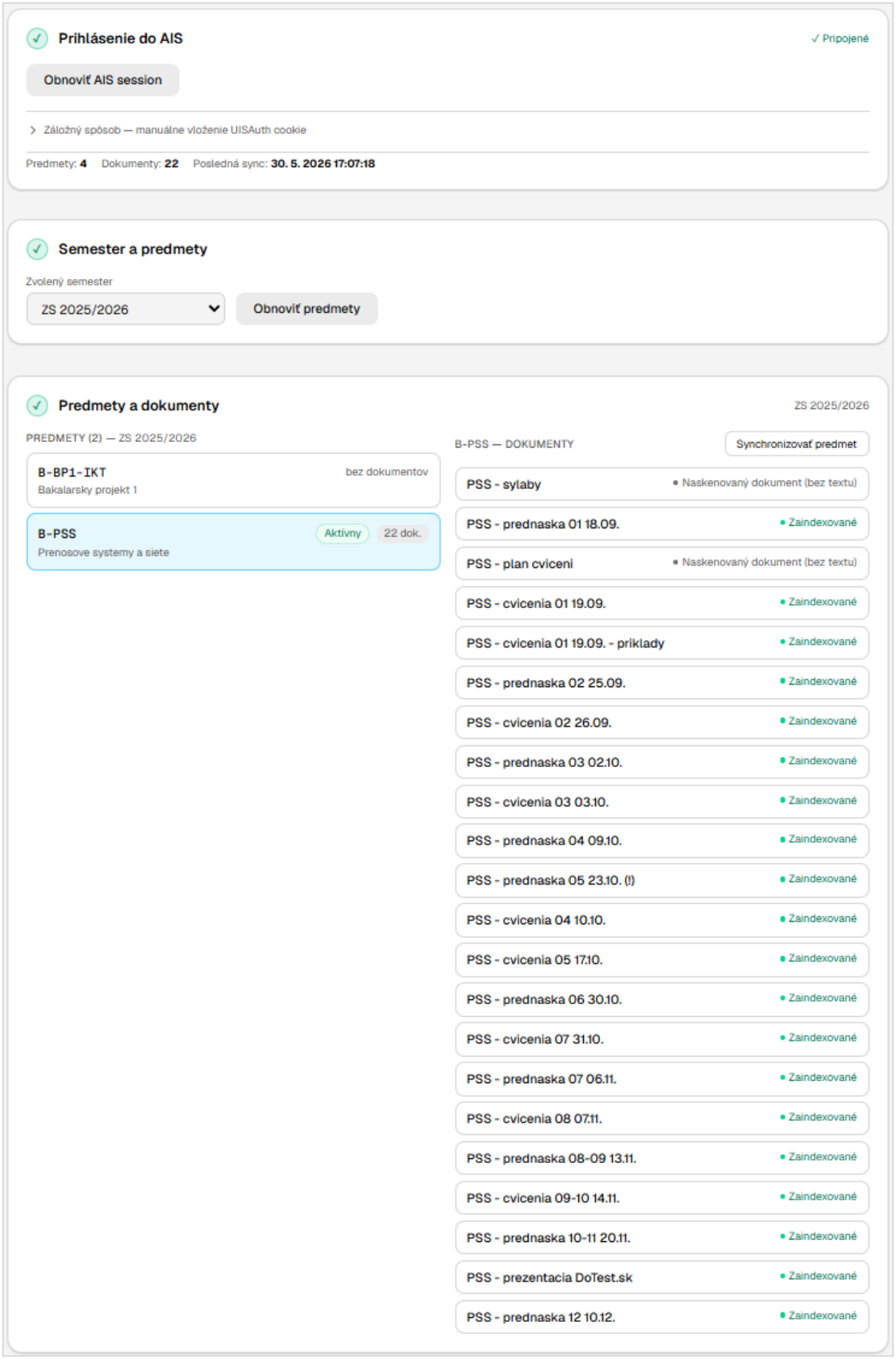
Pre vývojové prostredie je dostupná doplnková metóda prihlasovania prostredníctvom automatizácie prehliadača nástrojom Playwright. Táto metóda je výslovne vypnutá v produkčnom prostredí podmienkou NODE_ENV === 'production' v zdrojovom kóde.

Celkový tok integrácie s AIS, vrátane uloženia relácie používateľa, synchronizácie dokumentov a uloženia výsledkov do jednotlivých úložísk, znázorňuje Obr. 3.




*Obr. 3: Spracovateľský reťazec integrácie s AIS od uloženia relácie po synchronizáciu dokumentov*

Pohľad na rozhranie synchronizácie s AIS v systéme PortalAI po úspešnom pripojení je zobrazený na Obr. 4.




*Obr. 4: Rozhranie synchronizácie s AIS — výber semestra, zoznam predmetov a stav indexácie dokumentov*

## 3.4 Realizácia spracovania dokumentov a RAG komponentu

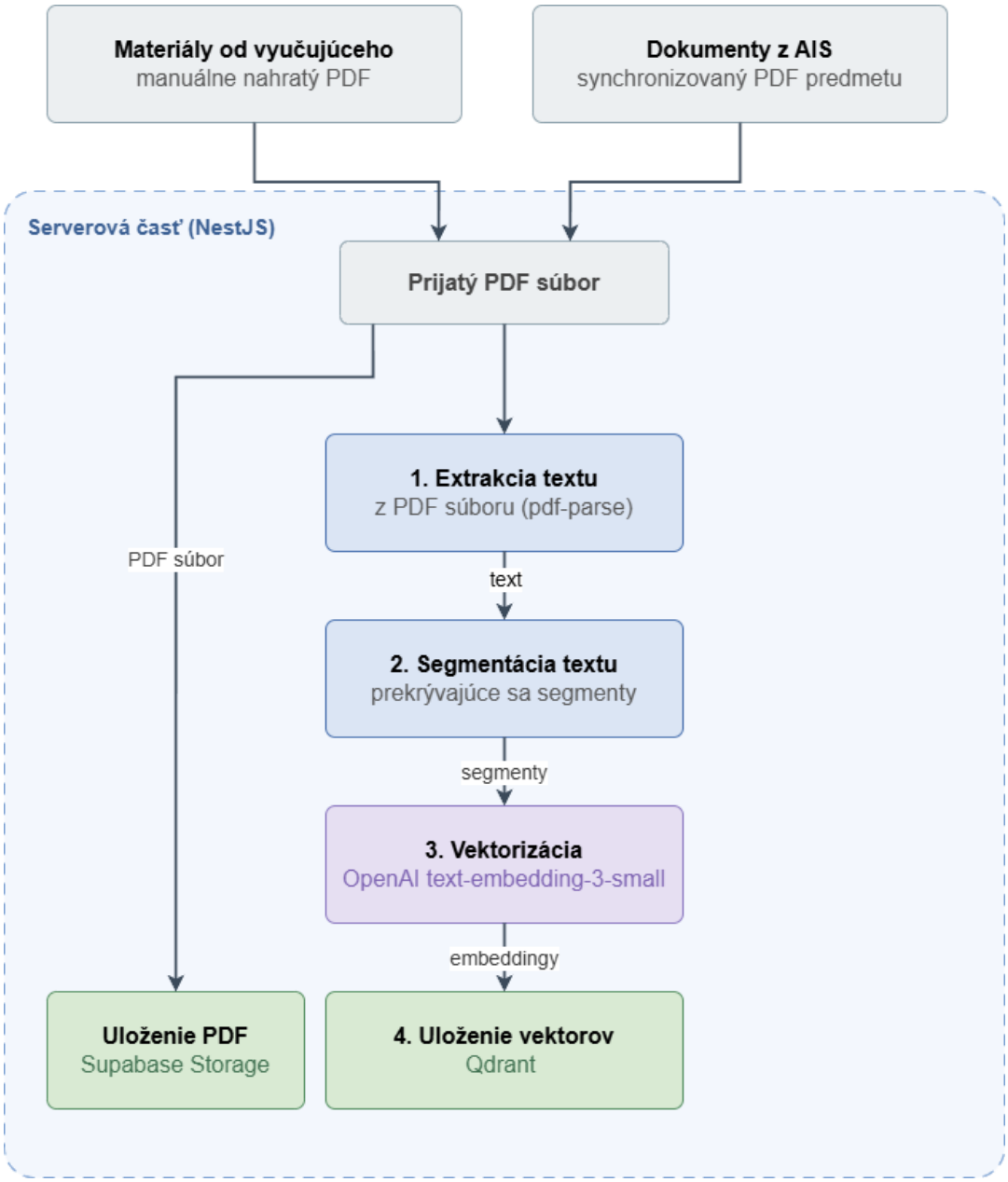
Cieľom spracovateľského reťazca dokumentov je pre obe domény rovnaký: textový obsah dokumentu sa extrahuje, rozdelí na segmenty, zakóduje do vektorov a uloží do kolekcie portalai_materials v Qdrant. Obe domény používajú rovnaký cieľový RAG princíp, no líšia sa poradím spracovania, evidovaním stavov a rozsahom metadát.

Spracovateľský reťazec dokumentov z AIS prebieha v nasledujúcom poradí. Dokument sa najprv stiahne zo servera AIS a uloží do Supabase Storage; stav záznamu AisDocument sa priebežne aktualizuje hodnotou syncStatus. Textový obsah sa extrahuje z PDF knižnicou pdf-parse. Text sa rozdelí na segmenty s veľkosťou 1 200 znakov a prekryvom 200 znakov, zakóduje do 1 536-rozmerných vektorov modelom text-embedding-3-small a vektory sa uložia do Qdrant. Hodnota contentHash — SHA-256 z binárneho obsahu súboru — sa uloží do záznamu AisDocument v relačnej databáze. Dokumenty v nepodporovanom formáte (nie PDF) sú označené stavom unsupported bez pokusu o extrakciu. Stav synchronizácie je sledovaný hodnotami syncStatus: uploaded (prechodný stav po nahraní súboru, pred extrakciou textu), failed_extraction, no_text, failed_indexing a indexed. Úplný prehľad hodnôt atribútu syncStatus je uvedený v časti 2.2.

Spracovateľský reťazec materiálov predmetov nahrávaných vyučujúcimi sa líši v poradí krokov a v absencii stavového sledovania. Pri materiáloch nahrávaných vyučujúcimi sa súbor najprv odovzdá knižnici pdf-parse na pokus o extrakciu textu. Následne sa súbor uloží do Supabase Storage a metadáta sa zapíšu do tabuľky materials v Supabase databáze. Ak sa z dokumentu nepodarí získať text použiteľný na segmentáciu, indexácia do Qdrant sa nevykoná. Následne prebehne segmentácia, vektorizácia a uloženie vektorov do Qdrant rovnakým spôsobom ako pri dokumentoch AIS. Tento tok neobsahuje sledovanie životného cyklu stavov syncStatus ani sledovanie contentHash. Vektory materiálov kurzu neobsahujú pole userId — izolácia prístupu je zabezpečená hodnotou courseId.

Postup spracovania dokumentov pre RAG komponent, od získania súboru až po uloženie vektorov do kolekcie Qdrant, znázorňuje Obr. 5. Podrobná štruktúra metadát uložených spolu s vektormi je opísaná v nasledujúcom texte.

Obrázok zobrazuje spoločné fázy spracovania dokumentu pre RAG; poradie uloženia súboru do úložiska sa medzi dokumentmi z AIS a materiálmi kurzov líši, ako je uvedené v texte.




*Obr. 5: Spracovateľský reťazec RAG od prijatia dokumentu po uloženie vektorov do Qdrant*

Štruktúra metadát vektora závisí od typu zdroja. Tieto metadáta slúžia predovšetkým na to, aby sa pri vyhľadávaní nemiešali vektory rôznych kurzov, predmetov z AIS a používateľov. Pre vektory materiálov kurzu (sourceType = "course_material") sú v payload na vrchnej úrovni uložené polia text, sourceType, materialId, courseId a qdrantVersion spolu s vnoreným objektom metadata. Tieto vektory neobsahujú pole userId — izolácia je zabezpečená hodnotou courseId, pretože materiály kurzu patria konkrétnemu kurzu spravovanému vyučujúcim. Pre vektory z AIS (sourceType = "ais") sú na vrchnej úrovni payload uložené polia text, sourceType, courseId, userId, subjectId a qdrantVersion spolu s vnoreným objektom metadata obsahujúcim napríklad pôvodný názov súboru. Pre AIS vektory slúži courseId ako chatCourseId príslušného záznamu AisSubject — nie ako identifikátor kurzu v relačnej databáze — a umožňuje konverzačnému asistentovi Flowise

vyhľadávať v kontexte správneho predmetu. Izolácia AIS dát je zabezpečená kombináciou sourceType + userId + subjectId.

Mazanie vektorov pri synchronizácii AIS sa vykonáva na úrovni predmetu filtrom sourceType + userId + subjectId — nie na úrovni jednotlivého dokumentu. Identifikátor bodu vo vektorovej databáze je deterministicky odvodený z hodnôt ako identifikátor predmetu, dokumentu a index segmentu prostredníctvom hashu SHA-1 formátovaného do tvaru UUID verzie 5. Identifikátor externého dokumentu (externalDocId) sa používa interne pri generovaní tohto deterministického identifikátora, ale nie je uložený v Qdrant payload ako samostatné filtrovateľné pole. Hodnotou contentHash dokumentu je SHA-256 z binárneho obsahu súboru, uložený v zázname AisDocument v relačnej databáze.

Vektory materiálov kurzu obsahujú v payload materialId, ktorý odkazuje na záznam v tabuľke materials v Supabase. Indexované polia sourceType, courseId, userId, subjectId a materialId sú registrované v Qdrant ako kľúčové slová, čo umožňuje rýchle filtrovanie pri vyhľadávaní.

Vektorová kolekcia portalai_materials je nakonfigurovaná s rozmerom vektora 1 536 a kosínusovou podobnosťou ako metrikou pre sémantické vyhľadávanie. Táto konfigurácia zodpovedá výstupnému rozmeru modelu text-embedding-3-small a štandardnej praxi pre semantické vyhľadávanie v embeddingovom priestore.

## 3.5 Realizácia kvízov a sledovania pokroku

Generovanie testovacích otázok prebieha nasledovne. Serverová časť overí, ktorý predmet je aktívny podľa záznamu activeAisSubjectId v AisSession — táto informácia sa čerpá výhradne z databázy. Systém odošle dva vektorové dopyty s rôznymi formuláciami (pri zadanej téme jeden cielený dopyt) a vyhľadá v Qdrant 20 až 30 najrelevantnejších segmentov filtrovaných podľa sourceType, userId a subjectId. Duplicitné body sa odstraňujú podľa identifikátora.

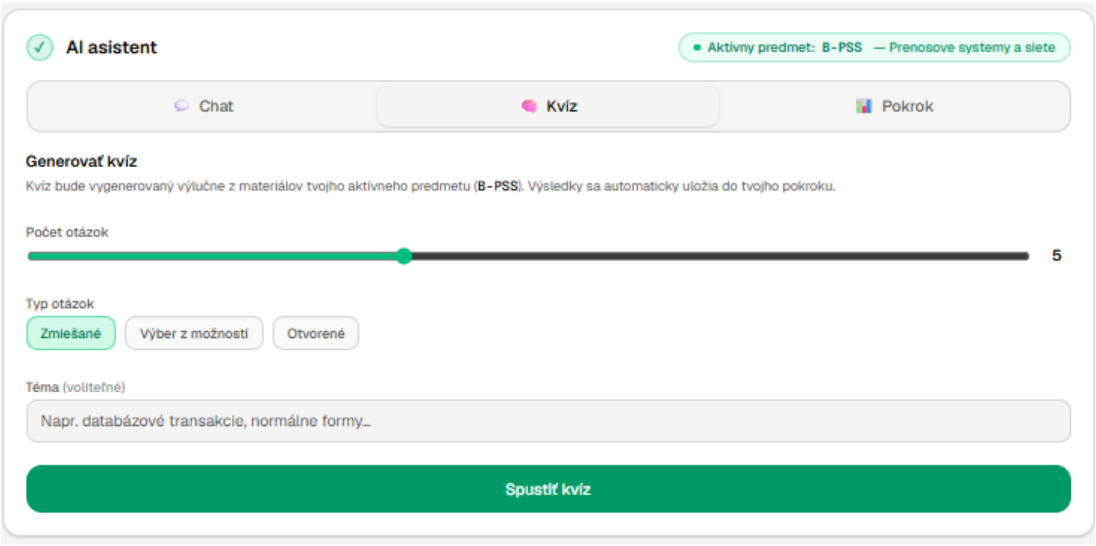
Zo získaných segmentov systém vyberie vyváženú podmnožinu: cieľový počet segmentov je max(count×3, 12). Systém následne vypočíta teoretický maximálny počet segmentov na zdrojový súbor ako podiel cieľového počtu a počtu unikátnych súborov, pričom výsledok zaokrúhli nahor. Na tento výsledok sa aplikuje dolná hranica dvoch segmentov na súbor — každý zdrojový súbor je tak zastúpený aspoň dvoma segmentmi aj v prípade, keď zdrojových súborov je veľa. Tým sa zaistí tematická rôznorodosť otázok aj pri predmetoch s viacerými zdrojovými dokumentmi. Výsledný kontext sa odovzdá modelu gpt-4o-mini, ktorý vygeneruje zmes otázok s výberom odpovede (štyri možnosti A–D) a otvorených otázok vo formáte JSON podľa pevnej schémy.

Hodnotenie odpovedí na otvorené otázky je realizované sémanticky prostredníctvom modelu gpt-4o-mini. Model hodnotí odpoveď na trojúrovňovej škále: správna (skóre 1), čiastočne správna (skóre 0,5) a nesprávna (skóre 0). Výstupom hodnotenia sú okrem skóre

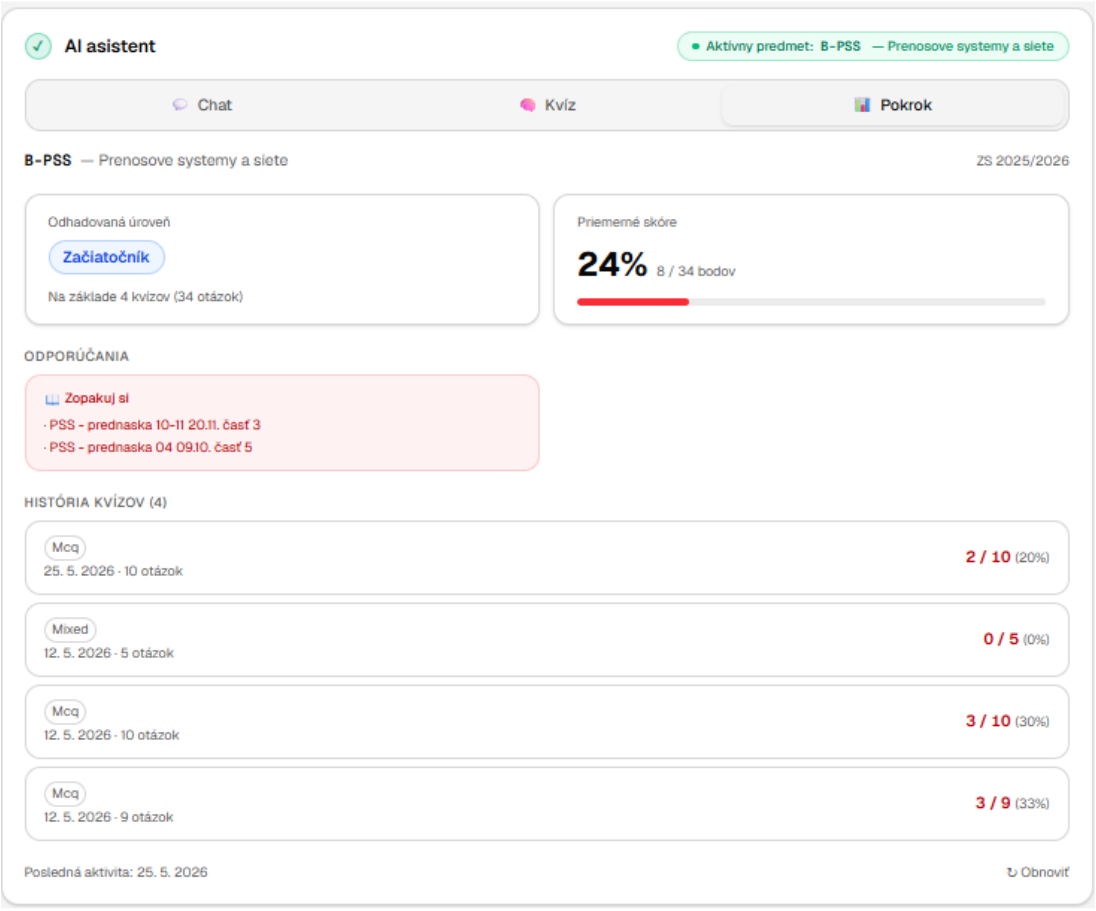
aj konkrétne prítomné a chýbajúce koncepty, ktoré sa zobrazujú študentovi ako spätná väzba. Model zohľadňuje synonymá a parafrázy — hodnotí sémantický obsah odpovede, nie doslovné znenie.

Sledovanie pokroku pracuje na dvoch úrovniach. Po každom ukončenom pokuse o kvíz serverová časť identifikuje témy, ku ktorým boli kladené otázky, a aktualizuje polia weakTopics a strongTopics v entite StudentCourseProgress alebo AisSubjectProgress. Témy sú identifikované podľa názvu zdrojového súboru. Zdrojový súbor je označený ako slabá téma, ak presnosť odpovedí súvisiacich s ním je pod 50 %; ako silná téma, ak presnosť dosahuje aspoň 80 %. Odhadovaná úroveň znalostí (beginner / intermediate / advanced) sa prepočítava z váženého priemeru skóre naprieč všetkými pokusmi, pričom váha zodpovedá počtu otázok v jednotlivých pokusoch. Rozhranie generovania kvízu a prehľad pokroku v učení sú zobrazené na Obr. 6.

(a) Rozhranie generovania kvízu




(b) Prehľad pokroku v učení (záložka Pokrok)




*Obr. 6: Rozhranie generovania kvízu a prehľad pokroku v učení v systéme PortalAI*

## 3.6 Realizácia konverzačného asistenta na platforme Flowise

Konverzačný asistent v PortalAI je postavený na platforme Flowise a využíva jej funkciu Agentflow. Agentflow sa od bežného chatového toku (Chatflow) líši tým, že

pracuje so zdieľanými stavovými premennými (Flow State) a tok môže vetvovať na základe ich hodnôt. Jeden tok tak dokáže obslúžiť požiadavky od rôznych typov používateľov bez potreby vytvárať samostatný Flowise tok pre každú rolu používateľa.

Každá požiadavka vstupuje do toku s vloženými kontextovými značkami. Proxy vrstva v Next.js pridá pred odoslaním do Flowise kontextové značky obsahujúce rolu používateľa, identifikátor používateľa a identifikátor kurzu alebo predmetu. Tri Custom Function uzly na začiatku toku tieto značky extrahujú regulárnymi výrazmi a zapíšu do Flow State premenných userId, courseId a userRole. Ak správa neobsahuje značku roly, hodnota userRole sa predvolene nastaví na student.

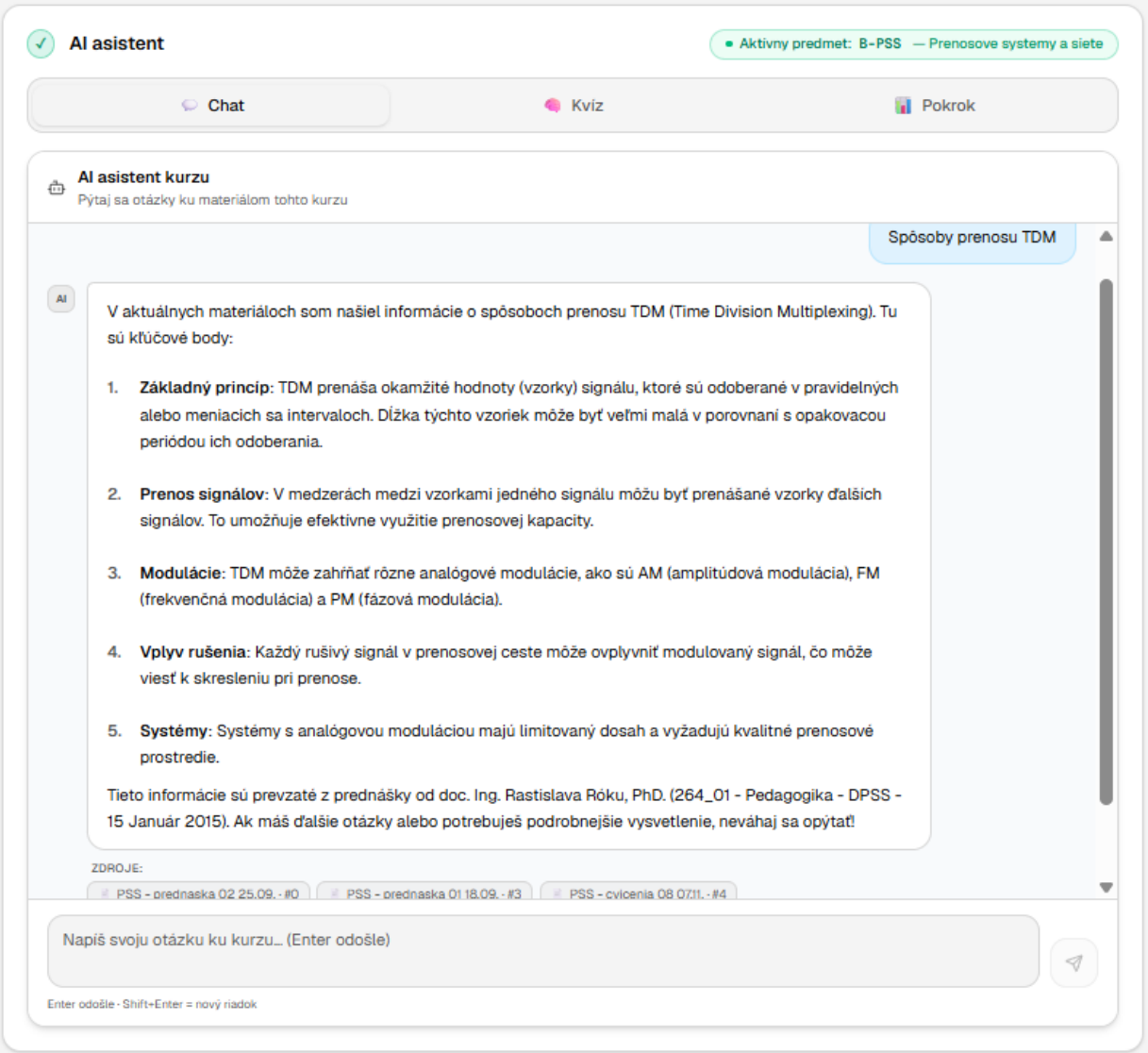
Celkový pohľad na to, ako jeden Flowise Agentflow extrahuje kontext požiadavky, vetví tok podľa roly používateľa a odovzdáva ho príslušnému špecializovanému agentovi, znázorňuje Obr. 7.




*Obr. 7: Vetvenie požiadaviek vo Flowise Agentflow podľa roly používateľa a typu otázky*

Tri extrakčné uzly typu Custom Function sú na obrázku pre prehľadnosť zobrazené ako jeden blok „Extrakcia kontextu“.

Príklad odpovede konverzačného asistenta s vypísanými zdrojovými dokumentmi je zobrazený na Obr. 8.




*Obr. 8: Konverzačný asistent v systéme PortalAI — odpoveď na otázku k obsahu predmetu so zdrojovými dokumentmi*

Názvy supervízorov a agentov uvádzaných ďalej (napríklad StudentSupervisor a TeacherSupervisor) sú implementačné názvy uzlov v prostredí Flowise; ich úlohou je smerovať otázku používateľa do správnej spracovateľskej vetvy. Prvý podmienený uzol rozvetví tok podľa roly používateľa určenej funkciou ROLE na dve vetvy. Študentské požiadavky spracúva uzol StudentSupervisor, požiadavky od vyučujúcich uzol TeacherSupervisor. Oba uzly sú jazykové modely nakonfigurované ako klasifikátory: z

textu požiadavky určia kategóriu, ktorá rozhodne, ktorý špecializovaný agent požiadavku nakoniec vybaví.

StudentSupervisor triedi požiadavky do troch kategórií. Kategória student_course aktivuje agenta StudentCourseAgent, ktorý prehľadáva kolekciu portalai_materials v Qdrant s kombinovaným filtrom. Filter hľadá v dvoch zdrojoch naraz: buď v materiáloch kurzu (sourceType = course_material, courseId = aktuálny kurz), alebo v dokumentoch synchronizovaných z AIS daného študenta (sourceType = ais, userId = aktuálny používateľ, courseId = aktuálny predmet). Kategória casual_chat odovzdá voľné konverzačné otázky agentovi CasualChatAgent, ktorý k vektorovej databáze vôbec nepristupuje. Kategória admin_guard blokuje požiadavky administratívneho charakteru.

Pole courseId v tomto filtri nesie hodnotu chatCourseId príslušného predmetu AIS; pri mazaní vektorov počas synchronizácie sa naopak používa subjectId. Obe hodnoty sa viažu na ten istý predmet, ale používajú sa v odlišných častiach systému.

TeacherSupervisor pracuje s piatimi kategóriami. Kategória teacher_course smeruje otázky k obsahu kurzu na TeacherCourseAgent. Kategória material_audit aktivuje MaterialAuditAgent, ktorý prehľadáva indexovaný obsah kurzu filtrom sourceType = course_material, courseId a umožňuje vyučujúcemu preskúmať, či nahrané materiály neobsahujú nezrovnalosti, nejasné pasáže alebo chyby. Kategória test_generator smeruje žiadosti o generovanie testov na TestGeneratorAgent; ten používa rovnaký model gpt-4o-mini ako ostatné špecializované agenty v toku. Kategórie casual_chat a admin_guard fungujú rovnako ako v študentskej vetve.

Všetky RAG agenty vracajú zdrojové dokumenty (Return Source Documents), takže rozhranie môže zobraziť, z akých materiálov odpoveď vychádzala. Nastavenie pamäti sa líši podľa agenta: CasualChatAgent uchováva celú históriu konverzácie, RAG agenty pracujú s pamäťovým oknom veľkosti 1, pretože kontext kurzu je vždy súčasťou požiadavky. Supervízorské uzly a TestGeneratorAgent pamäť nemajú — každé volanie je nezávislé.

## 3.7 Bezpečnostné obmedzenia realizovaného riešenia

Z hľadiska bezpečnosti je nevyhnutné súčasné riešenie integrácie s AIS charakterizovať ako prototypové kompromisné riešenie. Vychádza z vlastnej autentifikovanej relácie študenta v AIS, nie z trvalého servisného prístupu. Pri dlhodobom nasadení toto riešenie nie je ideálne a má nasledujúce konkrétne obmedzenia.

– Používateľ musí ručne skopírovať hodnotu súboru cookie z prehliadača. Tento úkon si vyžaduje technickú zdatnosť a zvyšuje riziko nesprávneho zaobchádzania s autentifikačným tokenom.

– Platnosť súboru cookie je viazaná na dobu aktívnej relácie AIS. Po jej vypršaní prestane synchronizácia fungovať; systém to zistí až pri pokuse o synchronizáciu.

– Závislosť na štruktúre HTML stránok AIS: zálohovací mechanizmus štyroch stratégií zmierňuje toto riziko, no neodstraňuje ho systémovo. Zmena štruktúry HTML na strane AIS môže narušiť funkčnosť.
– Šifrovanie cookie algoritmom AES-256-GCM na strane servera chráni uložené hodnoty v databáze pred neoprávneným čítaním aj pred ich pozmenením; bezpečnosť transportu závisí od konfigurácie HTTPS v produkčnom nasadení.

Ukladanie prihlasovacích údajov AIS (meno a heslo) v systéme sa z bezpečnostného hľadiska neodporúča: kompromitácia databázy by priamo ohrozila prihlasovacie údaje používateľov do inštitucionálneho systému. Takýto prístup nie je súčasťou realizovaného riešenia.

Realizovaný prototyp pracuje výhradne s reláciou používateľa, ktorý ju systému poskytne vedome; prípadné produkčné nasadenie by si však vyžadovalo schválenie takéhoto spôsobu integrácie zo strany inštitúcie.

Ideálnym dlhodobým riešením by bolo zavedenie štandardizovaného programového rozhrania na strane AIS alebo využitie inštitucionálneho mechanizmu delegovania prístupu (napr. OAuth alebo podobný protokol schválený inštitúciou). Takéto riešenie by eliminovalo závislosť na manuálnom kopírovaní cookie a prekonalo by prototypovú povahu súčasného prístupu. Realizácia by si vyžadovala inštitucionálnu spoluprácu.

## 3.8 Obmedzenia a ďalšie možné rozšírenia

Realizované riešenie má nasledujúce pretrvávajúce obmedzenia, ktoré neboli súčasťou aktuálnej implementačnej fázy:

– Chýbajúca rola administrátora: správa naprieč všetkými používateľmi a predmetmi nie je k dispozícii. Implementácia by si vyžadovala rozšírenie enumerácie Role v schéme Prisma, nové koncové body serverovej časti a administrátorské rozhranie.
– Absencia automatizovaných testov: automatizované testy s atrapami externých služieb neboli v tejto fáze implementované. Ich doplnenie je preto identifikované ako jedna z priorít ďalšieho rozšírenia systému.
– Chýbajúca infraštruktúrna dokumentácia: repozitár neobsahuje jednotný konfiguračný súbor docker-compose.yml pre spustenie celého systému PortalAI ani súbor .env.example. Nasadenie v novom prostredí vyžaduje manuálne nastavenie všetkých závislostí.
– Čiastočne implementované časti rozhrania: stránka /settings pokrýva lokálne preferencie, avšak nie úplnú editáciu serverového profilu; stránka /help obsahuje prehľad a FAQ, avšak nie podrobného sprievodcu nastavením AIS.

Z pohľadu technického rozvoja RAG komponentu sú perspektívnymi rozšíreniami hybridné vyhľadávanie (kombinácia hustého vektorového vyhľadávania s BM25, ktorú

Qdrant natívne podporuje od verzie 1.7) a štruktúrne segmentovanie pre pedagogické dokumenty. Predchádzajúca práca [1] ukázala, že pre informačné listy predmetov je účinnejšia segmentácia rešpektujúca hranice dokumentu. Analogicky by mohla byť užitočná segmentácia podľa hraníc odsekov alebo sekcií pri iných typoch pedagogických dokumentov.

# 4 Testovanie a skúsenosti z testovacej prevádzky

## 4.1 Metodika testovania

Testovanie systému PortalAI bolo realizované ako manuálne scenárové testovanie v lokálnom vývojovom a testovacom prostredí. Zvolená metodika vychádza z povahy systému: intenzívna integrácia s externými službami (AIS, Supabase, Qdrant, OpenAI API) a z toho vyplývajúca absencia napodobňovania externých služieb robia automatizované jednotkové testy pre integračné scenáre technicky náročnými.

Pre každý testovací scenár bola definovaná vstupná podmienka, vykonaná akcia a overenie výsledku porovnaním s očakávaným správaním. Testovanie nepokrylo záťažové ani bezpečnostné testovanie; merania výkonu sú orientačné a nie sú produkčnými referenčnými meraniami. Výsledky sú zdokumentované v nasledujúcich podkapitolách.

## 4.2 Funkčné testovanie

Tab. 2 zachytáva dvadsať manuálne overených testovacích scenárov. Testovanie prebiehalo v lokálnom vývojovom prostredí s reálnymi externými službami (Supabase, Qdrant, OpenAI API). Scenáre pokrývajú autentifikáciu, riadenie prístupu, správu kurzov, spracovanie materiálov, integráciu s AIS a kvízovú funkcionalitu.

*Tab. 2: Scenáre funkčného testovania*

| ID | Scenár | Očakávaný výsledok | Výsledok |
|---|---|---|---|
| T01 | Registrácia študenta | Konto vytvorené, študent sa úspešne prihlási | prešlo |
| T02 | Registrácia vyučujúceho | Konto vyučujúceho vytvorené so správnymi oprávneniami | prešlo |
| T03 | Prihlásenie s platnými údajmi | Cookie relácie nastavený, používateľ presmerovaný | prešlo |
| T04 | Prihlásenie s nesprávnym heslom | Prihlásenie odmietnuté, chyba zobrazená | prešlo |
| T05 | RBAC: študent pristupuje na stránku vyučujúceho | Prístup odmietnutý / presmerovanie | prešlo |
| T06 | Vyučujúci vytvára kurz | Kurz sa objaví v zozname kurzov vyučujúceho | prešlo |
| T07 | Vyučujúci nahrá PDF materiál | Súbor uložený a zaindexovaný do Qdrant | prešlo |
| T08 | Vyučujúci pridá študenta do kurzu | Študent vidí kurz vo svojom zozname | prešlo |
| T09 | Zoznam kurzov študenta | Študent vidí iba zapísané kurzy | prešlo |
| T10 | Konverzácia s asistentom v kurze | Asistent odpovedá na základe materiálov kurzu | prešlo |
| T11 | Generovanie kvízu kurzu | Kvíz vygenerovaný z materiálov kurzu | prešlo |
| T12 | Odovzdanie kvízu a aktualizácia pokroku | Výsledok uložený, pokrok aktualizovaný | prešlo |
| T13 | Pripojenie k AIS | Cookie AIS prijatý, semestre a predmety dostupné | prešlo |
| T14 | Načítanie semestrov a predmetov z AIS | Zoznam predmetov načítaný z AIS | prešlo |

| | | | |
|---|---|---|---|
| T15 | Synchronizácia dokumentov predmetu AIS | Dokumenty stiahnuté, extrahované a zaindexované | prešlo |
| T16 | Generovanie kvízu AIS | Kvíz vygenerovaný zo zaindexovaných dokumentov AIS | prešlo |
| T17 | Konverzácia s asistentom pre predmet AIS cez Flowise | Asistent odpovedá v kontexte aktívneho predmetu AIS | prešlo |
| T18 | História pokusov a pokrok | Predchádzajúce pokusy a pokrok zobrazené | prešlo |
| T19 | Odhlásenie | Cookie relácie vymazaný, presmerovanie na prihlásenie | prešlo |
| T20 | Vyučujúci odoberie študenta z kurzu | Kurz zmizne zo zoznamu kurzu študenta | prešlo |

Všetky dvadsať scenárov bolo úspešne overených. Počas testovania neboli zistené chyby, ktoré by bránili dokončeniu testovaných scenárov.

## 4.3 Výkonnostné merania

Výkonnostné merania boli vykonané manuálne v lokálnom testovacom prostredí. Čas trvania vybraných operácií bol odčítaný z panela Chrome DevTools Network ako trvanie príslušnej HTTP požiadavky alebo nadväzujúcej operácie. Každá operácia bola spustená a zaznamenaná trikrát. Cieľom merania nebolo vytvoriť produkčný benchmark, ale orientačne posúdiť používateľsky vnímanú odozvu najnáročnejších častí systému. Hodnoty preto zahŕňajú aj latenciu externých služieb, najmä OpenAI API, Supabase a Qdrant.

Namerané hodnoty vybraných operácií uvádza Tab. 3.

*Tab. 3: Výkonnostné merania vybraných operácií*

| **Operácia** | **Meranie 1** | **Meranie 2** | **Meranie 3** | **Priemer** |
|---|---|---|---|---|
| Synchronizácia predmetu AIS | 39,0 s | 38,3 s | 33,0 s | 36,8 s |
| Generovanie kvízu | 25,7 s | 20,9 s | 22,4 s | 23,0 s |
| RAG odpoveď konverzačného asistenta | 15,9 s | 7,8 s | 6,3 s | 10,0 s |

Synchronizácia predmetu AIS dosiahla priemer 36,8 s. Tento čas zahŕňa stiahnutie dokumentov z AIS, uloženie do Supabase Storage, extrakciu textu z PDF knižnicou pdf-parse, segmentáciu, volanie OpenAI Embeddings API pre každý segment a uloženie vektorov do Qdrant. Doba závisí od počtu a veľkosti dokumentov predmetu.

Generovanie kvízu dosiahlo priemer 23,0 s. Táto hodnota zahŕňa vektorové vyhľadávanie v Qdrant, konštrukciu kontextu a jedno volanie gpt-4o-mini so štruktúrovaným výstupom JSON. Variabilita je spôsobená rôznou dĺžkou generovaného výstupu podľa počtu a typu otázok.

RAG odpoveď konverzačného asistenta dosiahla priemer 10,0 s s výraznou variabilitou (6,3 s až 15,9 s). Variabilita je očakávaná, pretože latencia Flowise a jazykového modelu

závisí od zložitosti otázky, rozsahu získaného kontextu a dĺžky odpovede. Merania boli vykonané na lokálnej inštancii (localhost) a zahŕňajú latenciu externých služieb.

## 4.4 Skúsenosti z testovacej prevádzky

Spätná väzba od testujúcich používateľov bola získaná prostredníctvom anonymného dotazníka Google Forms. Dotazník vyplnili štyria študenti, ktorí systém PortalAI otestovali prostredníctvom verejného odkazu smerujúceho na lokálne spustenú vývojovú inštanciu autora. Cieľom bolo orientačné posúdenie použiteľnosti a užitočnosti systému z pohľadu študenta, nie reprezentatívna štatistická štúdia. Malý počet respondentov je pri testovacej prevádzke prototypu prirodzený a získané výsledky majú orientačný charakter.

Testujúci používatelia mali prístup k študentskej časti systému. Testovanie zahŕňalo orientáciu v rozhraní, prácu s kurzmi a materiálmi, kladenie otázok konverzačnému asistentovi, zobrazenie zdrojových dokumentov pod odpoveďami asistenta, generovanie kvízov a prehliadanie prehľadu pokroku v učení. Synchronizácia dokumentov s AIS nebola hlavnou súčasťou externého testovania — táto funkcia bola overená autorom v rámci manuálnych scenárov zdokumentovaných v časti 4.2, keďže vyžaduje cookie UISAuth z aktívnej relácie AIS.

Hodnotenie prebiehalo na päťstupňovej škále, kde hodnota 1 predstavovala najnižšie hodnotenie danej oblasti a hodnota 5 najvyššie hodnotenie.

*Tab. 4: Súhrn spätnej väzby študentov k systému PortalAI*

| **Hodnotená oblasť** | **Priemerné hodnotenie (1 – 5)** |
|---|---|
| Orientácia v systéme | 4,50 |
| Prehľadnosť hlavnej stránky a navigácie | 4,25 |
| Jasnosť umiestnenia predmetov, dokumentov, kvízov a AI asistenta | 4,00 |
| Prehľadnosť stránky predmetov alebo kurzov | 4,50 |
| Práca s dokumentmi predmetu | 4,00 |
| Užitočnosť konverzačného asistenta pri štúdiu | 4,25 |
| Zrozumiteľnosť odpovedí konverzačného asistenta | 4,50 |
| Relevantnosť odpovedí k obsahu predmetu | 4,25 |
| Prínos zobrazenia zdrojových dokumentov pod odpoveďou | 4,00 |
| Generovanie kvízu z materiálov predmetu | 4,25 |
| Zrozumiteľnosť otázok v kvíze | 4,25 |
| Spätná väzba po vypracovaní kvízu | 3,75 |
| Prehľad pokroku v učení | 4,00 |
| Celkové hodnotenie systému PortalAI | 4,50 |

Výsledky dotazníka naznačujú celkovo pozitívne hodnotenie. Najvyššie priemerné hodnotenie dosiahli orientácia v systéme, prehľadnosť stránky predmetov alebo kurzov, zrozumiteľnosť odpovedí asistenta a celkové hodnotenie systému, všetky na úrovni 4,50 z 5. Najnižšie hodnotenie zaznamenala spätná väzba po vypracovaní kvízu (3,75), čo naznačuje priestor na zlepšenie v tejto oblasti. Všetky hodnotené oblasti dosiahli priemer aspoň 3,75, čo naznačuje celkovo prijateľnú použiteľnosť systému v testovaných scenároch.

Z otvorených odpovedí vyplynulo, že testujúci za najužitočnejšiu funkciu považujú konverzačného asistenta so zobrazením zdrojových dokumentov pod odpoveďami — oceňovali predovšetkým to, že odpovede sú ukotvené v konkrétnych materiáloch predmetu a nie sú všeobecné. Viacerí testujúci kladne hodnotili aj generovanie kvízov z reálnych materiálov predmetu. Za menej jasné alebo ťažšie na použitie označili testujúci sekciu synchronizácie s AIS (nejasný účel a chýbajúca spätná väzba po kliknutí) a prehľad pokroku v učení (neistota, čo presne vyjadrujú zobrazené čísla). Jeden z respondentov mal ťažkosti s lokalizáciou záložiek kvízov a pokroku v rámci kurzu — čakal ich v bočnom menu namiesto záložiek v rámci stránky kurzu. Medzi konkrétnymi návrhmi na zlepšenie sa objavili: zachovanie histórie konverzácie po zatvorení stránky, podrobnejšia spätná väzba po kvíze s vysvetlením správnych odpovedí, vizualizácia vývoja pokroku v čase formou grafu a upozornenia pri pridaní nového dokumentu do kurzu. Jeden z respondentov zaznamenal výskyt podobných otázok pri opakovanom generovaní kvízu; ďalší uviedol, že zaškrtnutia z predošlého pokusu zostali viditeľné bez manuálneho obnovenia stránky.

Získaná spätná väzba má orientačný charakter. Dotazník vyplnili štyria testujúci, čo je príliš malá vzorka na štatisticky relevantné závery. Testovanie prebiehalo na vývojovej inštancii lokálne spustenej autorom a sprístupnenej prostredníctvom tunela, čo nemusí zodpovedať bežným podmienkam nasadenia. Napriek týmto obmedzeniam je spätná väzba cenná pre identifikáciu konkrétnych problémov s použiteľnosťou a pre prioritizáciu ďalšieho rozvoja.

Na základe získanej spätnej väzby možno orientačne uzavrieť, že systém PortalAI bol v rámci testovania hodnotený ako zrozumiteľný a použiteľný. Pre ďalší rozvoj je vhodné zamerať sa na spresnenie vysvetlenia synchronizácie s AIS priamo v rozhraní, podrobnejšiu spätnú väzbu po kvíze, vizualizáciu pokroku v čase a zachovanie histórie konverzácie s asistentom.

## 4.5 Zhrnutie testovania

Manuálne scenárové testovanie pokrylo dvadsať testovacích prípadov zahŕňajúcich autentifikáciu, riadenie prístupu na základe rolí, správu kurzov, nahrávanie a indexáciu materiálov, integráciu s AIS, generovanie a vyhodnocovanie kvízov a sledovanie pokroku. Všetky scenáre boli úspešne overené.

Výkonnostné merania ukázali, že najnákladnejšou operáciou je synchronizácia predmetu AIS (priemer 36,8 s), čo je podmienené počtom externých volaní a veľkosťou spracovávaných dokumentov. Generovanie kvízu (23,0 s) a RAG odpoveď konverzačného asistenta (10,0 s v priemere) sú pre interaktívne použitie akceptovateľné, hoci v produkčnom prostredí by mohlo byť vhodné implementovať asynchrónne spracovanie náročnejších operácií.

Výsledky testovania tak potvrdzujú, že systém je v kontrolovanom testovacom prostredí použiteľný v hlavných študentských a vyučujúcich scenároch. Pretrvávajúce obmedzenia systému — absencia automatizovaných testov, chýbajúca rola administrátora a prototypová povaha integrácie s AIS — sú identifikované a zdokumentované v časti 3.8.

Spätná väzba od štyroch testujúcich študentov zaznamenaná v časti 4.4 potvrdila celkovo pozitívne hodnotenie systému, pričom celkové hodnotenie dosiahlo priemer 4,50 z 5. Testujúci ocenili predovšetkým konverzačného asistenta so zdrojovými dokumentmi a generovanie kvízov z materiálov predmetu. Identifikované podnety na zlepšenie sa týkajú prehľadnejšieho vysvetlenia synchronizácie s AIS, podrobnejšej spätnej väzby po kvíze, vizualizácie pokroku v čase a zachovania histórie konverzácie s asistentom.

# Záver

Táto bakalárska práca naplnila obe úlohy zo zadania. Prvá úloha — vykonanie analýzy aktuálneho riešenia a návrh zmien — je riešená v kapitole 2. Druhá úloha — realizácia, testovanie a zozbieranie skúseností z testovacej prevádzky — je zdokumentovaná v kapitolách 3 a 4, pričom realizácia upraveného riešenia je opísaná v kapitole 3 a testovanie v kapitole 4.

Systém PortalAI predstavuje výrazný posun oproti predchádzajúcej práci [1]. Technologický zásobník bol zjednotený do jazyka TypeScript na oboch stranách systému; softvérové rámce Next.js a NestJS poskytujú dobrý základ pre ďalší rozvoj. Vektorová databáza Qdrant s relačnou databázou PostgreSQL a úložiskom Supabase tvoria dátovú vrstvu s relačným modelom, riadenými migráciami a samostatným vektorovým úložiskom, ktorá lepšie zodpovedá potrebám rozšíreného prototypu. Systém bol rozšírený o nové oblasti funkcionality: integráciu s akademickým informačným systémom STU, automatickú synchronizáciu dokumentov predmetov, generovanie testovacích otázok, ich sémantické hodnotenie a mechanizmus sledovania pokroku v učení na úrovni tém zdrojových dokumentov.

Implementovaná integrácia s AIS umožňuje automaticky synchronizovať dokumenty zapísaných predmetov za aktuálny semester. Jej prototypová povaha — závislosť na manuálne skopírovanom autentifikačnom súbore cookie a na štruktúre HTML stránok AIS — je identifikovaným obmedzením, nie chybou návrhu. Riešenie vychádza z absencie verejne dostupnej dokumentácie k programovému rozhraniu AIS; pri dlhodobom nasadení si vyžaduje inštitucionálnu spoluprácu.

Manuálne testovanie potvrdilo funkčnosť systému v dvadsiatich scenároch, vrátane registrácie, autentifikácie, RBAC, správy kurzov, nahrávania a indexácie materiálov, synchronizácie AIS, generovania a vyhodnocovania kvízov a sledovania pokroku. Výkonnostné merania v lokálnom testovacom prostredí ukázali priemerné časy 36,8 s pre synchronizáciu predmetu AIS, 23,0 s pre generovanie kvízu a 10,0 s pre RAG odpoveď konverzačného asistenta.

Oblasti vyžadujúce ďalšiu prácu zahŕňajú absenciu automatizovaných testov, chýbajúcu rolu administrátora, neúplnú infraštruktúrnu dokumentáciu a prototypovú povahu autentifikácie voči AIS. Tieto obmedzenia nebránia správnej funkcii systému v kontrolovanom prostredí, ale je potrebné ich riešiť pred nasadením do produkčnej prevádzky. Tri smery ďalšieho rozvoja sú z technického hľadiska perspektívne: hybridné vyhľadávanie v Qdrant, štruktúrne segmentovanie pedagogických dokumentov a inštitucionálna integrácia s AIS nahradzujúca súčasný prístup cez autentifikačný súbor cookie.

# Literatúra


[1] PETKES, M. Portál pre študentov s AI pomocníkom. Bakalárska práca. Bratislava: Fakulta elektrotechniky a informatiky, Slovenská technická univerzita v Bratislave, 2025.

[2] LEWIS, P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv preprint arXiv:2005.11401. 2020. Dostupné online: https://arxiv.org/abs/2005.11401

[3] VASWANI, A. et al. Attention Is All You Need. In: Advances in Neural Information Processing Systems. 2017, roč. 30. Dostupné online: https://arxiv.org/abs/1706.03762

[4] GAO, Y. et al. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv preprint arXiv:2312.10997. 2024. Dostupné online: https://arxiv.org/abs/2312.10997

[5] WANG, S. et al. Large Language Models for Education: A Survey and Outlook. arXiv preprint arXiv:2403.18105. 2024. Dostupné online: https://arxiv.org/abs/2403.18105

[6] NestJS. Dokumentácia. [online]. 2025. [cit. 2025-05-24]. Dostupné online: https://docs.nestjs.com

[7] Next.js. Dokumentácia. [online]. 2025. [cit. 2025-05-24]. Dostupné online: https://nextjs.org/docs

[8] Qdrant. Dokumentácia. [online]. 2025. [cit. 2025-05-24]. Dostupné online: https://qdrant.tech/documentation

[9] OpenAI. Text Embedding Models. [online]. 2025. [cit. 2025-05-24]. Dostupné online: https://platform.openai.com/docs/guides/embeddings

[10] Prisma. Dokumentácia ORM. [online]. 2025. [cit. 2025-05-24]. Dostupné online: https://www.prisma.io/docs

[11] Supabase. Dokumentácia. [online]. 2025. [cit. 2025-05-24]. Dostupné online: https://supabase.com/docs

## Deklarácia použitia umelej inteligencie

Pri vypracovaní tejto bakalárskej práce boli použité nástroje generatívnej umelej inteligencie v súlade s Opatrením č. 1/2024-O „Používanie umelej inteligencie na Slovenskej technickej univerzite v Bratislave“. Konkrétne boli použité:

– OpenAI ChatGPT (2026): jazyková a štylistická kontrola vybraných častí textu;

– Anthropic Claude (2026): jazyková kontrola vybraných častí textu, úprava kódu a úpravy štylistiky;

– DeepL Translator (2026): preklad, jazyková kontrola vybraných častí textu.